

PROJECT: Market Data Feed Handler Logbook

Roark Menezes

21/10/25

PREFACE

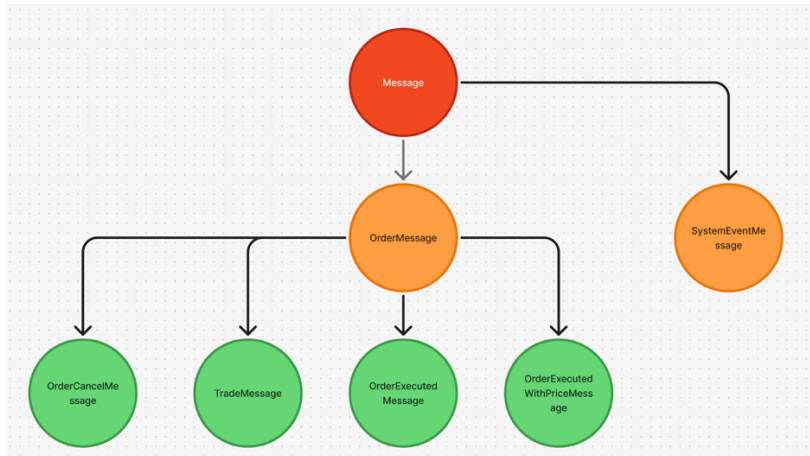
This project was completed with the main goal being to test the limits of all my C++, operating systems, networking and computer architecture knowledge to this point in time. The aim here is not to just complete the project, but to design a solution that is elegant, readable and most importantly **time and memory efficient**. I have completed this project in the same frame of mind that I believe a low latency C++ developer would. While completing this project, I have made note of areas where I can perform optimization and have planned it so that there will be **many versions of the project**, each with an increasing number of optimizations. I then plan to perform **benchmarking** on each to compare their performance. This is a passion project of mine as I have always wanted understand how important each clock cycle is to price information and trading strategy. This form of extreme optimization for ultra low latency operation is what gives one firm a profitable edge over another, to truly understand this importance I have decided to complete this project.

21/10/25

Today I am beginning work on the project, starting with some reading up on the ITCH protocol. I will create a server that generates ITCH messages wrapped in UDP packets and sends it to a multicast IP on my local network. I will then create a UDP multicast receiver to read the UDP messages. This will all be done on **C++ in Linux**. The **ITCH protocol** is an application layer protocol (L5-7 in the OSI model) that breaks down market data into a stream of bytes, especially designed for low latency environments such as high frequency trading systems. The variable length binary messages are usually sent over **UDP multicast**, although TCP unicast can be used for extra data separate from the original market data stream (such as the Glimpse server in ASX's implementation). UDP does not provide any reliability of delivery, however within each ITCH message is a sequence number, which the receiver can use to ascertain correct order of messages. Any missing messages can be retrieved from a re-request server, given the sequence number.

The protocol implementation varies from exchange to exchange, and thus the way the bytes are parsed changes as well. But the general specification is similar, each has a specific **message type** (1 byte ASCII character) followed by a **variable length payload**. Theoretically UDP has a maximum size of $65535 - 8 = 65527$ bytes (16 bit length in 8-byte UDP header) but this is also bound by the IPv4 maximum, so considering the 20-byte IP header (and 16 bit total length field) we get 65507 bytes. This is also bound by the MTU of the link on which the packets will be transmitted, so for my purposes the max size will be limited to a 1500-byte MTU (running **ifconfig** and finding the wifi NIC name shows mtu value of 1500), which gives me a max UDP payload size of $1500 - 20 - 8 = 1472$ bytes, and thus an ITCH payload size of **1471 bytes with a 1 byte message type preceding it**.

Subscription to the real exchange ITCH byte stream is for authenticated vendors only so I decided to simulate it. I created a few classes, taking advantage of inheritance and polymorphism with C++ (base Message and OrderMessage class, derived TradeMessage and other order classes) to help organise simulated market data for **adding orders, executing orders, executing orders with a price, system events, cancelling orders and trades** (only a subset of those available in the real ITCH).



ITCH Message Class Inheritance Diagram

I've also created a UDP socket and set it up to send traffic to my preconfigured multicast socket address (using `inet_pton`). I named this program the **itch-message-generator (IMG)**. I have only simulated sending 'A' ITCH messages (add order) which are of size 48 bytes (without any pragma pack, < 64 bytes will fit on the same cache line so speed of memory is not an issue, but it does introduce unnecessary network overhead. Additionally, I haven't enabled threads yet so false sharing won't be an issue, although I will have to change this later once implementing multithreading). After setting up the entire system to send traffic to a multicast IP, at a **cadence of 1 second** I ran tcpdump on my WiFi NIC (wlp1s0), while simultaneously running IMG and I could see all the multicast traffic showing up at 239.1.1.1:30001 (the IP/port pair I selected).

```

roarkmenezes@dredge: ~
roarkmenezes@dredge:~$ sudo tcpdump -i wlp1s0 dst host 239.1.1.1 -v -n
tcpdump: listening on wlp1s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
11:42:31.026859 IP (tos 0x0, ttl 1, id 24553, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:32.029308 IP (tos 0x0, ttl 1, id 25554, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:33.029928 IP (tos 0x0, ttl 1, id 25794, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:34.030478 IP (tos 0x0, ttl 1, id 25835, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:35.031002 IP (tos 0x0, ttl 1, id 26363, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:36.031551 IP (tos 0x0, ttl 1, id 26432, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:37.032184 IP (tos 0x0, ttl 1, id 26935, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:38.033911 IP (tos 0x0, ttl 1, id 27350, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:39.034158 IP (tos 0x0, ttl 1, id 27471, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:40.034738 IP (tos 0x0, ttl 1, id 28244, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48
11:42:41.035255 IP (tos 0x0, ttl 1, id 28543, offset 0, flags [DF], proto UDP (17), length 76)
192.168.4.174.53983 > 239.1.1.1.30001: UDP, length 48

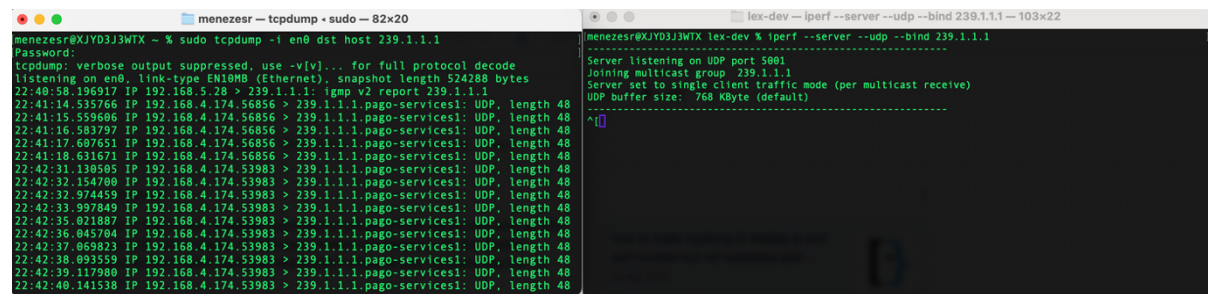
```

UDP packets sent to multicast IP loopback

I did not expect my wlp1s0 interface to pick up the multicast traffic, since I have not sent any **IGMP membership report** messages and subscribed to that multicast IP. But I learned that the Linux kernel enables multicast traffic loopback by default. I decided to test if another device on my local network could pick up the traffic, so I subscribed to the multicast group 239.1.1.1 (by running `iperf --server --udp --bind 239.1.1.1`). This triggers the Linux network stack to send IGMP membership messages to the WiFi access point (which it uses to build its **IGMP snooping table**) via the wifi NIC. The NIC

then adds a mapping for this to its' filter, and so any frames that arrive destined for this address (**01:00:5e:01:01:01** MAC equivalent) are stored in the DMA, ready for access by the Linux kernel.

Now, when I run IMG on my original device, the frames are sent along the network segment to my WiFi access point, as normal, and now the AP has learned from snooping that it should forward these packets to my other device. I can see this by running `tcpdump` on my other device and see the packets coming in (including the first IGMP membership report message)



```
menezesr@XJY03J3W7X ~ % sudo tcpdump -i en0 dst host 239.1.1.1
Password:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on en0, link-type EN10MB (Ethernet), snapshot length 524288 bytes
22:40:58.196917 IP 192.168.5.28 > 239.1.1.1: igmp v2 report 239.1.1.1
22:41:14.535766 IP 192.168.4.174.56856 > 239.1.1.1.pago-services1: UDP, length 48
22:41:15.559686 IP 192.168.4.174.56856 > 239.1.1.1.pago-services1: UDP, length 48
22:41:16.583797 IP 192.168.4.174.56856 > 239.1.1.1.pago-services1: UDP, length 48
22:41:17.607651 IP 192.168.4.174.56856 > 239.1.1.1.pago-services1: UDP, length 48
22:41:18.631671 IP 192.168.4.174.56856 > 239.1.1.1.pago-services1: UDP, length 48
22:42:31.138585 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:32.154788 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:32.974459 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:33.997849 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:35.021887 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:36.045704 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:37.069823 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:38.093559 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:39.117980 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48
22:42:40.141538 IP 192.168.4.174.53983 > 239.1.1.1.pago-services1: UDP, length 48

menezesr@XJY03J3W7X lex-dev % iperf --server --udp --bind 239.1.1.1
Server listening on UDP port 5801
Joining multicast group 239.1.1.1
Server set to single client traffic mode (per multicast receive)
UDP buffer size: 768 KByte (default)
^[[
```

Second device joins multicast group, receives same packets

24/10/25

Today I planned to focus on finishing implementation of the other ITCH messages but I realised that my current implementation was not serializing data, which meant extra padding was being added to my byte stream (although it can still fit on a single cache line, it introduces network congestion that can be avoided).

If I was to use a virtual function to implement serialization in the base Message class, each derived class would need to store a **vtable** (in .rodata), to help perform virtual function lookups on each call to the virtual function. A **vp**tr variable is also stored in each object of the class, and so this would add an 8 byte overhead (vptr), a vtable lookup and **an indirect function call** (because the vtable holds only a function pointer). This is what is known as a **control hazard** because the CPU must then dereference the pointer, which can takes time. You would think the CPU would need to stall while it resolves the pointer, however to avoid this the CPU can use **branch prediction** to guess the result, without waiting for the dereference to finish. This is done using the branch target buffer (**BTB**) which stores prediction data based on branch address (branch name), target address (predicted address) and prediction info (such as a 2 bit saturating counter). If the guess is right, there are no problems and execution continues, the BTB is updated to reflect a successful guess, but if it is not right, the entire pipeline, including the return order buffer (ROB, mechanism that allows out of order execution while ensuring in order commit in the CPU) must be flushed, which is detrimental to performance. This is why I will avoid virtual functions altogether, the aim here is for simulating a **low latency environment** as much as possible.

Instead of virtual functions, I chose to use a **templated base class** that has a method which statically resolves to a method from the derived class (**static polymorphism**). This resolves at compile time, meaning no vtable, no vptr, no indirect function calls, much more lightweight than virtual functions. For my use case, that meant updating the Message and OrderMessage classes to be templated and have a serialize method that calls the derived Message/OrderMessage class serialization methods. For static polymorphism to work, each base class needs to know the derived type, that is why I templated both the Message and OrderMessage classes (refer to inheritance diagram above showing base classes). **This gives me polymorphism for my classes without any virtual function overhead.**

```
template<typename DerivedMessage>
class Message {
public:
    char        m_messageType;
    uint64_t    m_timestamp;           // We use 8 bytes here, but 6 bytes when
serializing
public:
```

```

    Message(char messageType, uint64_t timestamp): m_messageType(messageType),
m_timestamp(timestamp){};
    Message() = default;
    size_t serialize(uint8_t *buf) const {
        return static_cast<const DerivedMessage*>(this)->implSerialize(buf);
    }
};

```

Base Message class with static polymorphism (serialize method)

I then moved on to implement the derived classes' serialization methods, **implSerialize**, which each write the relevant message types' attributes in bytes to a byte array (uint8_t buffer[]) and returns the numbers of bytes written. This is used for the **sendto()** function in the main thread, which takes in the buffer written to and the size. I also added a couple templated helper functions to convert all multi-byte integers (timestamp, order reference number etc.) to **network byte order** (MSB) which is the standard for all network protocols, and thus applied to ITCH. Running the code now with the serialization additions, I can see that the **TradeMessage()** message is sending as 32 bytes instead of the previous 48. **That 16 bytes saved over billions of repetitions will make a huge difference.**

```

2 errors generated.
● menezesr@XJYD3J3WTX itch-message-generator % g++ -std=c++20 main.cpp messages.cpp helper.cpp -o itch_gen
● menezesr@XJYD3J3WTX itch-message-generator % ./itch_gen
Setup socket and dest addr succesfully.
Sent bytes: 32
48 bytes sent. timestamp: 45398879680000
2624344023 of SOL order added
Sent bytes: 32
48 bytes sent. timestamp: 45399880196000
166544036 of NVDA trade executed
Sent bytes: 32
48 bytes sent. timestamp: 45400885407000
3934233180 of VET order added
Sent bytes: 32
48 bytes sent. timestamp: 45401896573000
2580176442 of GOOGL trade executed
Sent bytes: 32
48 bytes sent. timestamp: 45402901920000
1066861179 of NVDA trade executed
^C
● menezesr@XJYD3J3WTX itch-message-generator %

```

Updated ITCH Message Generator showing 32 byte TradeMessage sent

I plan to run benchmarking tests on my code once the implementation is complete to see where optimizations can be made.

26/10/25

Today I decided to focus on **randomly generating the ITCH messages**. After some investigation I decided the best way would be to initialize an **array of function pointers** that is randomly indexed each loop iteration. Each function pointer references a function that writes the serialized version of a specific message type to the given buffer and returns the number of bytes written. This is given as inputs to `sendto()`, sending the correct amount of bytes over the network.

With the way I have implemented it, my `generateMessage()` function is calling a random element from the function pointer array. This results in an **indirect function call**, meaning the branch prediction unit (**BPU**) will constantly suffer mispredicted branches, because the BTB will most likely store incorrect guesses (there is no pattern to the array access it is inherently random). One method of optimization here is to use a **switch statement**. Although it is less elegant, it would be faster because each function call can be placed inline, allowing the **compile time computation of the addresses** the `generateMessage` function must jump to next, rather than doing arithmetic with registers (e.g. `call [rdi + rax*8]`) i.e. no branch prediction necessary.

With the `generateMessage()` code added, I ran the program once more and also ran `tcpdump` for 239.1.1.1 in parallel (on a separate device, which I also ran `iperf` on to subscribe to the multicast IP). I also added overload functions for the `<<` operator so the messages sent can be logged to `stdout` on the server device.

```
-----
STARTING ITCH MESSAGE GENERATOR...
-----
INITIALISED SOCKET
-----
SET SOCKET OPTIONS SUCCESSFULLY
-----
Setup socket and dest addr succesfully.
-----
STARTING GENERATOR
-----
timestamp: 21508500102000
MARKET OPEN

Sent bytes: 8
timestamp: 21509505349000
905001223 shares from order no: 1002 cancelled.

Sent bytes: 19
timestamp: 21509590767000
3268524166 of META order added

Sent bytes: 32
timestamp: 215091592522000
45791209 shares from order no: 1005 executed.

Sent bytes: 19
timestamp: 21509259794000
2791313596 of VET order added

Sent bytes: 32
timestamp: 2150936000892000
4030054055 shares from order no: 1007 cancelled.

Sent bytes: 19
timestamp: 215094605540000
3424269600 of XRP trade executed

Sent bytes: 32
timestamp: 215095610660000
1551934074 shares from order no: 1009 executed.

Sent bytes: 19
timestamp: 215096611580000
1239540483 shares from order no: 1003 cancelled.

Sent bytes: 19
```

Random ITCH messages sent to 239.1.1.1


```

summermenezes — iperf -s -u -B 239.1.1.1 - 80x24
cs0, cs1, cs2, cs3, cs4, cs5, cs6, cs7, ef, le, nqb, nqb2, ac_be, ac_bk, ac_vi,
ac_vo, lowdelay, throughput,
reliability, or a numeric value.

The TCP window size option can be set by the environment variable
TCP_WINDOW_SIZE. Most other options can be set by an environment variable
IPERF_<long option name>, such as IPERF_BANDWIDTH.

Source at <http://sourceforge.net/projects/iperf2/>
Report bugs to <iperf-users@lists.sourceforge.net>
summermenezes@C02DW5TNQ05D ~ % iperf -s -B 239.1.1.1
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
summermenezes@C02DW5TNQ05D ~ % iperf -s -u -B 239.1.1.1
-----
Server listening on UDP port 5001
Joining multicast group 239.1.1.1
Server set to single client traffic mode (per multicast receive)
UDP buffer size: 768 KByte (default)
-----

summermenezes — tcpdump -s 80x24
16:28:06.981654 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 8
16:28:08.005783 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 8
16:28:09.029757 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 19
16:28:11.077570 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:12.101632 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 19
16:28:13.125649 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 8
16:28:14.149585 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:15.178607 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 8
16:28:15.997914 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 8
16:28:17.021775 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 19
16:28:18.040830 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:19.069740 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:20.088699 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 8
16:28:21.112708 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:22.136677 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:23.160824 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:24.189653 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:25.213738 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 19
16:28:26.032993 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:27.051876 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 32
16:28:28.076381 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 19
16:28:29.104828 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 19
16:28:30.123941 IP 192.168.5.28.49156 > 239.1.1.1.pago-services1: UDP, length 8

```

Subscribing to multicast IP on another device + random ITCH messages received

The above shows the **random ITCH messages being sent from the server and received on the client (both distinct devices)**, we can see each message received now has a size specific to the message type. I added some extra logging macros to help with logging crucial debug data.

The problem now is that I am introducing **network congestion** by not bunching up the messages before sending. Each packet introduces an 8 byte UDP header and 20 byte IPv4 header, leaving **1472 byte total payload size**. I am currently using only a maximum of 32, meaning minimum **1440 bytes is going to waste**. One point of optimization here is to extend the server to bunch up data before sending. This would require simulating something close to how **Nagle's algorithm** is implemented in TCP, where messages are only sent if there is no in-flight data or at least 1 SMSS worth of data is present in the sending buffer. Using a **separate thread that produces ITCH messages and writes them to the share buffer and having the main thread consume (read from and run sendto()) would also speed up execution**. I will leave this for another day, but as said previously I am logging all these optimizations so I can come back and implement them later.

I also noticed that after removing the 1 second cadence, a lot of data was not being received on the client side. **Real exchanges add sequence numbers to the ITCH messages, so clients can re-request messages they did not receive**, but I have not yet implemented this (although it is in the plan). This is what a client could use to measure their packet loss rate, allowing them to make changes accordingly. For now I will reintroduce the 1 second cadence for debugging, I will bring the simulation closer to reality once all the working parts are in order.

29/10/25

Today I began work on the **market-data-feed-handler (MDFH)** which aims to read multicast traffic (from the same 239.1.1.1 IP) and parse the messages correctly. The setup will consist of **IMG** running on one device and **MDFH** on the other. I had to study up on C++ socket programming a bit here, it wasn't as simple as the server, due to requiring multicast group membership. The aim here was to create a socket that binds to a socket that receives the correct multicast traffic. The kernel is then able to deliver the UDP payload (IP and UDP header both stripped) to the socket receive buffer. But how does it know which socket to deliver the bytes to? To answer this question I consulted the linux man page on the `setsockopt()` function and found that a multicast membership request can be sent via a socket option at the IP layer. It is this action that prompts the kernel to send an IGMP membership report packet to the default gateway (IP address converted to the internal IP of the multicast enabled interface on the router, TTL set to 1 so it never leaves the network) or the switch uses IGMP snooping to log it in its snooping table. In either case, the L3/L2 device can forward multicast traffic back to the receiver correctly because of this IGMP report. The kernel knows exactly which socket set that option, and can correctly enqueue multicast packets in that socket's receive buffer. Now lets go into the exact steps I executed to create the mfdh.

I created a socket descriptor with the **SO_REUSEADDR** and **SO_REUSEPORT** option (even though generally you want a single socket per feed per process, I plan to have another process bind to this port for analytical purposes). I then set the binding information, using **htons** and **htonl** for the port and multicast IP (**exact same as the server send side**). I then ran the `bind()` command on this socket address (`sockaddr_in`) and then subscribed to the multicast IP on the IP of interface I expect the multicast frames to end up on (which for the device I am on currently is the **en0** interface with IP 192.168.5.28). I then created a small `uint8_t` buffer and tried to **recv()** from the socket, but nothing was coming through. I ran **netstat -g** before and after running the process and could see the 239.1.1.1 IP coming up in the multicast group memberships, so something was probably wrong at the socket layer. I then noticed the issue was with the address I was binding to. Using the multicast IP to bind to makes no sense, because multicast IPs do not belong to specific NIC's. So it is like saying "accept all traffic that is received on the 239.1.1.1 local IP" but NICs are not assigned multicast IPs. The NIC just converts a certain multicast IP membership request into a multicast MAC address, which it uses to compare incoming multicast frames. It does not behave like unicast frames. So I had to change the binding address to **INADDR_ANY** instead.

```
// Set binding information
sockaddr_in listen_addr{};
listen_addr.sin_family = AF_INET;
listen_addr.sin_port = htons(PORT);
```

```
listen_addr.sin_addr.s_addr = htonl(MULTICAST_IP); // This makes no sense, we
should bind to all IPs, allowing the multicast traffic to flow through because of
the membership addition later
```

```
// BIND
if (bind(sockfd, (sockaddr*)&listen_addr, sizeof(listen_addr)) < 0) {
    perror("bind");
    close(sockfd);
    exit(EXIT_FAILURE);
}
```

Code snippet for socket binding for multicast traffic

I then followed this with the **IP_ADD_MEMBERSHIP** option to tell all the NIC's (or in our case, a specific one we know the multicast frames are going to be received on) to add an entry to its multicast filtering table (once this table length is exceeded, usually 64 entries, it switches to hash-based filtering which uses a bitmap stored in a hardware register, but running **netstat -g** shows that this is nowhere near capacity).

```
// Join the 239.1.1.1 multicast group
ip_mreq mcastMship{};
mcastMship.imr_multiaddr.s_addr = inet_addr(MULTICAST_IP); // Convert the
multicast IP to network byte order
mcastMship.imr_interface.s_addr = inet_addr(NIC_IP); // Apply the multicast
filtering at NIC we know the multicast frames will reach (can also use
htonl(INADDR_ANY) if we want to apply to ALL NICs

// Set the IP_ADD_MEMBERSHIP socket option, using the ip_mreq struct
if (setsockopt(sockfd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mcastMship,
sizeof(mcastMship)) < 0)
{
    perror("Failed to set IP_ADD_MEMBERSHIP option at IP layer (IPPROTO_IP) on
client socket.\n");
    return 1;
}
```

Code snippet for joining multicast group

Compiling and running mfdh now, shows the exact amount of bytes sent by itch-gen being received. At this point, I have a barebones client that can read in the multicast UDP traffic as a stream of independent messages (each an ITCH message).

01/11/25

The focus of today was parsing the messages received in mfdh into the different types of ITCH messages. Because we know the exact type based on the first byte of each message, I can build a unique parser for each of the 6 message types, **add order** ('A'), **trade** (non-displayable order, 'P'), **order executed** ('E'), **order executed with price** ('X'), **system event** ('S') and **order cancelled** ('C') messages. I initially thought to setup structs to copy the serialized bytes into, but this turned out to be a problem because the serialized data was packed and thus unaligned, reading it directly into a struct packed using **pragma pack(push, 1)** would mean the data would be misaligned. **Misaligned data is a huge problem when optimizing for low latency**, because it essentially means the CPU's load/store unit requires 2 memory accesses for. Consider an 8 byte integer (64 bit system) that begins at an address ending in 0x03 and ends at an address 0x0C. The LSU is configured to access according to an alignment, so in this case it would need to fetch each of the 8 bytes over which this integer is spread and then perform another operation to combine the 2, which means a total of 3 operations. **Over millions of operations, this nanosecond of latency will prove detrimental to a trading strategy.**

Additionally the same principle can apply to loading data/instructions into the cache. **Having data not 64 byte aligned means the 2 fetches would need to apply**, and this also introduces problems such as cache pollution as the CPU may need to load data from each cache line that it may not even use. Thus, I had to come up with another solution. As optimization is the name of the game, I instead tried to parse the data directly from the input buffer, running a custom parsing function based on the first byte read (the message type), using basic pointer arithmetic to read the fields. I also used **alignas(64)** on the input buffer to ensure that it is 64 byte aligned and the CPU can load an entire ITCH message into the cache for quick parsing. Because each ITCH message is at most 32 bytes (in my implementation) each iteration of the reading loop will access the same 64 byte portion of the buffer, which is already in the cache. This **cache hot** implementation means the kernel can very quickly access the buffer and copy the data from the socket buffer into the user space buffer. There is still the **double context switching** of user space to kernel space (to copy the data into the user space buffer) and then back into user space, but this has been added to the running optimization list for later. **Proper profiling will be done later perf.**

One more area of optimization is to do with the flow of the data from receiving at the NIC to entering the user space buffer. The kernel accesses the DMA buffer (based on interrupts by the NIC or polling) and moves the data into the socket receive buffer. Then, whenever an recv() call is made, the kernel again takes over and copies that data into the user space buffer. We can instead **bypass** the socket receive buffer copy, and **directly copy from the DMA into user space**, however this will be done later.

Continuing with the struct parsing idea, the plan now is to have separate structs for each data type (using similar inheritance to IMG) and then log the data to the console based on the parsed message. This will be just for demonstration purposes, in a real setting the structs would probably be removed entirely, and the parsing would happen directly from the buffer (e.g. reading the quantities, order ids etc. directly into an order book). I created a family of structs, with the same layout as the classes from the server. To avoid the earlier found issues with branch prediction, I used a **switch statement in my parser loop** that checks the first byte of every message (which has the messageType) and jumps to the relevant parsing function. This way, I get direct branches, which allows the BHT to get skewed toward the correct values, allowing the CPU to guess correct nearly every time and use the correct jump target from the BTB.

Once I setup the parsing loop, I moved on to implement the parsing functions. Each function had a similar layout to the server side implementation, but this time it was the opposite (i.e. I was setting the struct values from the buffer instead of setting the buffer from the struct values). One problem I encountered here was **parsing the timestamp**. It was serialized as a **6 byte value**, but I needed to parse it into an 8 byte struct member. To do this, I created a helper function that initialises an 8 byte unsigned int (uint64_t) to 0 and then set only bytes 3 to 8 from the buffer, meaning the top two bytes are 0. I then performed the network to host conversion (ntohll(), opposite of htonll() however I used my own implementation on the server side as I wasn't aware of htonll()) and could see the correct nanosecond values of the timestamps coming out (exactly what was sent on the server side). This works because the number of nanoseconds in a day is $8.64e+13$, which easily fits in a 48 bit number.

```
// Safely read 6-byte timestamp (network byte order) into aligned uint64_t
inline uint64_t readTimestamp(const char* buf, size_t &offset) {
    uint64_t tmp = 0;
    // first 2 MSBs are empty (all 0), then store the timestamp value
    std::memcpy(std::bit_cast<char*>(&tmp) + 2, buf + offset, 6); // upper 6 bytes,
    I will always use bit cast instead of reinterpret cast because it is safer, as it
    uses a constexpr length comparison (compile time) and doesn't violate C++ strict
    aliasing rules
    offset += 6;
    return ntohll(tmp); // convert to host byte order
}
```

Function to read a 6 byte value into an 8 byte struct member

The next problem I faced was how to efficiently display this nanosecond value as a human readable timestamp (HH:MM:SS.NS). My first thought was to manually format it using a series of if else statements that would print the hours, minutes, seconds and nanoseconds based on division and modulus operations. It was from researching this that I came across a term known as **branchless programming**. In the name of optimization, avoiding branches where possible is key. This is why I removed all my conditional statements and used a series of fixed statements that provides

determinism for each instruction. In the conditional case, each incorrect prediction would cause a complete pipeline flush, including registers, ROB, reservation stations and pipeline registers, which over millions of iterations can be the cause of missed trades and market data. I have kept both versions of the function, so I can perform proper profiling later once the basics of itch-gen and mfdh are complete.

I also created similar helper parsing functions for the 8 byte orderRefNumber and 4 byte shares and prices values across each message type. I then also overloaded the << operators to give me human readable logs to check for any differences between server side and client side output.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

LISTENING FOR MULTICAST TRAFFIC ON 239.1.1.1
*[06:57:20.027577000] | MARKET OPEN*

[06:57:21.032954000] | Order executed with price Order ID: [1001]: 1185689103 @ 1110716945
[06:57:22.035036000] | Order Added: [1010]: 2029888318 of $DOGE to Sell @ 3215836241
[06:57:23.040417000] | Order cancelled Order ID: [1006] cancelled.
[06:57:24.045782000] | Order executed: [1002]: 3738972695 shares
[06:57:25.051143000] | Order cancelled Order ID: [1005] cancelled.
[06:57:26.056516000] | Order executed with price Order ID: [1010]: 3472941516 @ 4267167702
[06:57:27.060721000] | Order cancelled Order ID: [1006] cancelled.
[06:57:28.064496000] | Order executed: [1005]: 1160439454 shares
[06:57:29.069861000] | Order cancelled Order ID: [1005] cancelled.
[06:57:30.075207000] | Order executed: [1005]: 2387632041 shares
[06:57:31.076892000] | Order cancelled Order ID: [1008] cancelled.
```

MFDH output showing readable timestamp, order ids, shares, prices, tickers and message types

I also added a second logging mode for full debug level, but this made the code a bit unreadable, so I created a logging class to alleviate this issue. The logging class was simple, it just has a log level that dictates whether to use verbose output or raw output. I kept the << operator overloads as the **VERBOSE** output, but the problem I faced was how to implement the other **RAW** output. I initially thought to have a template function that calls a getRawLog() function on the input message type, so that would require creating a virtual function in the base Message class, and having each derived message implement it. But again the aim here is to reduce indirect execution as much as possible, so I opted for a CRTP implementation of my message classes to allow for static polymorphism instead of the slower dynamic polymorphism of virtual functions.

I created a getRawLog() function in the base Message class, that applies a static_cast to the current object and instead runs its own implementation of the getRawLog() function, called getRawLogImpl() (**the same as how I implemented CRTP in the itch message generator**). I applied the const qualifier where appropriate to assert that I would not be changing anything (as is expected of a logging class), and added the log levels as an enum type. The result was a robust Logger class with templated logging

functions. It can easily extend to have more levels and uses static polymorphism avoiding the runtime latency associated with dynamic dispatch.

```
LISTENING FOR MULTICAST TRAFFIC ON 239.1.1.1
[X] timestamp=24019318782000 orderRefNumber=1010 executedShares=3614780728 executedPrice=3816915990 printable=Y
[E] timestamp=24020329590000 orderRefNumber=1009 executedShares=1051030997
[X] timestamp=24021340106000 orderRefNumber=1010 executedShares=1453986545 executedPrice=2188294426 printable=N
[P] timestamp=24022350361000 orderRefNumber=1010 shares=3356627884 stock=MSFT buySell=S price=1813615892
[X] timestamp=24023360852000 orderRefNumber=1001 executedShares=2836054685 executedPrice=1393610632 printable=Y
[X] timestamp=24024371441000 orderRefNumber=1007 executedShares=3186492725 executedPrice=2825473402 printable=N
[E] timestamp=24025379522000 orderRefNumber=1009 executedShares=1183186727
[C] timestamp=24026387983000 orderRefNumber=1006 cancelledShares=3617685876
[X] timestamp=24027397842000 orderRefNumber=1005 executedShares=2385310538 executedPrice=2679632852 printable=N
[P] timestamp=24028406169000 orderRefNumber=1005 shares=1803501313 stock=GALA buySell=S price=2474385530
[P] timestamp=24029414512000 orderRefNumber=1005 shares=846162283 stock=GOOGL buySell=B price=3843525560
[S] timestamp=24030421195000 eventCode0
[E] timestamp=24031429544000 orderRefNumber=1007 executedShares=2219709558
[E] timestamp=24032437858000 orderRefNumber=1006 executedShares=2710660814
[X] timestamp=24033446207000 orderRefNumber=1008 executedShares=2426705412 executedPrice=207051621 printable=N
[E] timestamp=24034448204000 orderRefNumber=1004 executedShares=1275789165
```

MFDH output showing raw parsed messages

With a more robust logging output, I can better understand how to plan out the storing of this data in the kdb database later. One more point for improvement I noticed is that each call to `recv()` is reading only a single message, which means **each message received is causing by a context switch** (to the kernel to copy the data from the socket receive buffer into the user space buffer). This can be improved by using the **batched reads mechanism of `recvmsg()`**, so the syscall is only initiated after a batch of UDP packets are waiting in the socket receive buffer. I leave this for another day. One more quick log I want to add is for computing the **one-way latency**, which can be done by finding the current time in nanoseconds since midnight and taking the timestamp value from the ITCH messages away from this (pretty much the same function I used in the itch message generator, it's a simulation of the TSOPT TCP header option used for protection against wrapped sequence numbers, PAWS, and for RTT/RTTVAR calculation, which is essential in setting the RTO). After adding this, the output I get is as below:

```
[E] timestamp=42140230738000 orderRefNumber=1010 executedShares=329793408 | 62249000 ns
[E] timestamp=42141235842000 orderRefNumber=1005 executedShares=2121999448 | 81131000 ns
[X] timestamp=42142240437000 orderRefNumber=1006 executedShares=806457835 executedPrice=3936832481 printable=N | 100539000 ns
[E] timestamp=42143245940000 orderRefNumber=1002 executedShares=2398094234 | 119029000 ns
[X] timestamp=42144248790000 orderRefNumber=1004 executedShares=172226049 executedPrice=3653045121 printable=Y | 140220000 ns
[X] timestamp=42145254114000 orderRefNumber=1002 executedShares=3987026371 executedPrice=976680009 printable=N | 158976000 ns
[E] timestamp=42146255467000 orderRefNumber=1002 executedShares=2249337638 | 18446744073686271616 ns
[P] timestamp=42147257163000 orderRefNumber=1002 shares=1703171856 stock=SOL buySell=B price=4255701511 | 18446744073708364616 ns
[E] timestamp=42148258648000 orderRefNumber=1007 executedShares=2547652153 | 21579000 ns
[A] timestamp=42149264128000 orderRefNumber=1009 shares=1001714268 stock=META buySell=B price=4289852032 | 39957000 ns
[E] timestamp=42150265528000 orderRefNumber=1006 executedShares=159934486 | 62709000 ns
[A] timestamp=42151267071000 orderRefNumber=1009 shares=250698011 stock=ETH buySell=S price=2930760717 | 85133000 ns
[A] timestamp=42152272584000 orderRefNumber=1010 shares=3560590083 stock=META buySell=S price=184042829 | 103500000 ns
[C] timestamp=42153273976000 orderRefNumber=1003 cancelledShares=3191636655 | 126260000 ns
```

Latency from object creation on server to parsing on client

I can see that most values are at 100-500ms, but there are some outliers there that have nonsensical values (such as the SOL buy order above) but at this stage I am not sure what is causing those, so more investigation is required.

04/11/25

Now that general parsing of the ITCH feed is working, I decided to begin on a few items on the optimization list, beginning with batching ITCH messages before sending (to achieve peak network utilization, i.e. as close to the 1472 byte payload max as possible). As I was brainstorming the best way to go about doing this, I thought of something I learned while learning about data flow in TCP, which was the **bandwidth delay product (BDP)** which is indicative of the optimal use of a TCP connection. When the congestion window is equal to the BDP, then one can conclude the network is at optimal utilization (thus it forms the upper bound for the congestion window). It is usually found by increasing congestion window (through initial TCP slow start phase) until RTO expiry or high RTT variance occurs (which is processed by TCP as data loss). When that happens, the congestion window is set to half the current value (sssthresh) and it enters congestion avoidance.

With my implementation, I want to do something similar but just not as complex. I want to find a value of my **“UDP congestion window”** (which I know isn’t real, but is just another name I use for the threshold after which **IMG’s** send buffer is flushed and all messages in there are sent) so that minimal network losses are incurred. As this is a multicast connection, there is no point finding a dynamic value as different clients will have different forms of packet loss, so I will tune for a static value. **But finding the equilibrium between context switch costs** (when switching to kernel mode and sending the ITCH message via the multicast interface) **and packet loss on the network can be done by benchmarking**, which is what I will do to find the best value for the buffer size. I will also need to implement a **timer that flushes the buffer**, which covers the case where messages may not pass the send threshold and may begin aging.

My first thought was to update IMG to copy ITCH messages into the user-space sending buffer until the capacity is reached. But I realised that this would be wasteful, there is no point copying bytes into the ITCH message buffer and then again into the user-space send buffer, it just wasted CPU cycles. So instead, I decided to have a **unified buffer**, and remove the ITCH message buffer so we would write directly to the user-space send buffer (only if the write would mean the capacity is NOT exceeded, otherwise the buffer is flushed). This requires tracking the location within the send buffer to next write to (reset when buffer flushed).

Then I ran into another issue which was that I **need to know the length of the message prior to writing into the buffer. But how can I do this?** I thought the best way would be to pass in the current bytes remaining value (capacity – current position) and check if the next computed message’s length exceeds that. If it does, then we should not write and instead flush the buffer, which can be done by passing -1 as a return value and check this in the main sending loop. **But the problem now is that the generated message is now lost**, and on the next loop iteration a brand new message will be

generated. How can we retry writing the same message after flushing the buffer? I thought we could have a retry buffer written to when the next message to write would exceed the capacity of the send buffer. Then in the main sending loop, we would check if this buffer is set and if so then copy this retry buffer to the send buffer first. But this introduces copies and branch prediction logic, **that will introduce latency and pipeline flushes respectively when the prediction is wrong. I decided to analyse the tradeoffs here.** The new maximum ITCH message size is 24 bytes, and with a 1472 byte payload that gives me 61 ITCH messages. That means the branch predictor will **mis-predict once every 61 iterations, 1.6% frequency.** Thus I have learned about the trade-offs in building low latency programs. If I opt for this solution I will amortize the cost of the sendto() syscall, by spreading it out over 61 iterations, but 1 of those iterations has a higher cost than the others because it involves a mispredicted branch, which will flush the pipeline. **But how costly is the sendto() call?**

I created a small program to measure this, creating a dummy socket and then timing how long it would take to run 100,000 iterations of a sendto() call to that socket (addr doesn't matter), and then finding the average. Running the program 5 times, I found that the average cost of a single sendto() call (found the average of each of the 5 averaged values) is approximately **8444.49 nanoseconds**. I am running this on an M1 Pro chip, which features up to 3.2GHz, which means 3.2 cycles per nanosecond. That is a delay of **27022.368 cycles**. If this was happening every single iteration that would be detrimental to overall performance, but I am amortizing its' cost so it only happens once every 61 iterations.

I then also wrote a program to **measure the cost of a pipeline flush**, as I could not find any information online. The program measures the runtime of 2 loops with branches in them. The first has a branch that always evaluates in one direction (and uses a volatile int value so that the compiler does not optimize the code and completely remove the loop). The second loop has a branch that always mis-predicts every certain number of iterations (which means it will incur a pipeline flush cost that many iterations). We can then time each loop, the second must be greater than the first, and so we can find their difference giving us the total cost of all pipeline flushes. We then divide this by the total number of mispredictions (which we control) to get the cost of each flush. Running this program 10 times, I get an average value of **5.92953 nanoseconds**, which equates to roughly **19 cycles per pipeline flush**.

This means that the cost of a pipeline flush is nothing compared to the time saved from amortizing the `sendto()` syscall over 61 iterations, and so we can conclude that using this retry buffer is a relatively good choice.

Measurement	Average time (ns)	Clock cycles
<code>sendto()</code> syscall	8444.49	27022
pipeline flush	5.92	19

Table showing `sendto()` and pipeline flush costs

The cost of the pipeline flush is negligible compared to `sendto()`, even though we have 2 of these: one to check if the retry buffer is populated and one for checking if the next message would exceed the send buffer capacity. **The total cost is still negligible.**

The conclusion of this analysis led me to begin implementing the retry buffer. I implemented a retry buffer struct that contains a `uint8_t` array and a size (of ITCH message to retry). The struct is created in the main thread, and then in the each iteration of the main sending loop, the `generateMessage()` function call was updated to take in a pointer to this retry buffer and a const reference to the amount of bytes remaining. In that function, the retry buffer was written to only if the generated message's size exceeds the bytes remaining (which means we cannot write it to the main sending buffer). It also returns 0 in this case, which is what the if statement in the main sending loop checks (what we benchmarked earlier).

```
constexpr ssize_t SEND_BUFFER_SIZE = 1472; // MAX UDP PAYLOAD SIZE

alignas(64) uint8_t sendBuf[1536];
alignas(64) retryBuffer retryBuf;
ssize_t pos = 0;
while(1) {
    ssize_t len = generateMessage(sendBuf + pos, retryBuf, SEND_BUFFER_SIZE -
pos);
    pos += len;
    if (!len) {
        // Flush the buffer
        ssize_t bytesToSend = pos - len;
        ssize_t sent = sendto(sockfd, sendBuf, bytesToSend, 0, (sockaddr*)
&dest_addr, sizeof(dest_addr));
        if(sent < 0) {
            perror("Could not send data");
        } else {
            std::cout << "Sent bytes: " << bytesToSend << std::endl <<
std::endl;
        }
        memcpy(sendBuf, retryBuf.buf, retryBuf.size);
        pos = len;
    }
}
```

```

        fflush(stdout);
    }

```

Code snippet of updated sending loop

```

ssize_t generateMessage(uint8_t *buf, retryBuffer &retryBuf, const size_t
&bytesRemaining) {
    std::mt19937 gen(std::random_device{}());
    std::uniform_int_distribution<uint32_t> dist(0, NUM_MESSAGES-1);
    uint32_t chosenGenerator = dist(gen);

    size_t msgLen = generators[chosenGenerator](buf);
    if (msgLen > bytesRemaining) {
        memcpy(retryBuf.buf, buf, msgLen);
        retryBuf.size = msgLen;
        return 0;
    }
    return msgLen;
}

```

Code snippet of updated generateMessage() function

Then I ran the tcpdump command on the client side (after also running iperf to join the multicast group) and I could see the packets coming in. **I decided to also properly test my SEND_BUFFER_SIZE maximum value** (set to 1472 as this is the max UDP payload size as calculated earlier) to see if it was really at its maximum value. I added the -vv flag to tcpdump, to show the IP packet data as well, and from there I could **inspect the fragmentation parts of the header (which is the 13 bit offset, 16 bit fragment ID and 3 bit flags)**. I decided to try sizes that would get me 2 or 3 fragments, and below is the output.

For 2 fragments I used a 1473 UDP payload size. What I expect is 2 fragments, the first with an L2 payload size of 1500 bytes (20 byte IPv4 header + 1480 byte IP payload, made up of a 8 byte UDP header + 1472 bytes of the UDP payload) AND 21 bytes (20 byte IPv4 header and a 1 byte IP payload, made up of the remaining 1 byte from UDP payload, UDP header is not repeated). The offset of the first fragment is 0, and I should expect the MF bit in the IP header to be set, and the second fragment should have a 1480 byte offset (first fragments IP packet is 1480 bytes) and no flags set.

For 3 fragments, I would need something larger than 3000 bytes L2 payload, which means more than 1472 + 1480 (8 byte UDP header is only in first fragment) = 2952 bytes UDP payload. I used 2953. Now I expect to see 2 x 1500 byte fragments and a 21 byte fragment. The first and second should both have a 20 byte IP header, 1480 byte payload and MF flag set, but the first is just made up of the 8 byte UDP header and 1472 byte UDP payload with offset of 0, while the second has a full 1480 byte UDP payload as the IP payload. And an offset of 1480. The third should have a 20 byte IP payload and the remaining 1 byte UDP payload as the IP payload with an offset of 2960

Running these different setups for the SEND_BUFFER_SIZE (and comparing with the base 1472 value), I get the outputs below (tweaked the main sending loop so it would wait until exactly these many bytes (1472, 1473 and 2953) are to be sent.

```

tcpdump: listening on en0, link-type EN10MB (Ethernet), snapshot length 524288 bytes
12:18:58.419733 IP (tos 0x0, ttl 1, id 49715, offset 0, flags [+], proto UDP (17), length 1500)
    172.20.10.13.49789 > 239.1.1.1.pago-services1: UDP, length 1473
12:18:58.419754 IP (tos 0x0, ttl 1, id 49715, offset 1480, flags [none], proto UDP (17), length 21)
    172.20.10.13 > 239.1.1.1: udp
12:19:09.260662 IP (tos 0x0, ttl 1, id 46367, offset 0, flags [+], proto UDP (17), length 1500)
    172.20.10.13.64867 > 239.1.1.1.pago-services1: UDP, length 2953
12:19:09.261813 IP (tos 0x0, ttl 1, id 46367, offset 1480, flags [+], proto UDP (17), length 1500)
    172.20.10.13 > 239.1.1.1: udp
12:19:09.261817 IP (tos 0x0, ttl 1, id 46367, offset 2960, flags [none], proto UDP (17), length 21)
    172.20.10.13 > 239.1.1.1: udp
12:19:28.681315 IP (tos 0x0, ttl 1, id 38897, offset 0, flags [none], proto UDP (17), length 1500)
    172.20.10.13.56928 > 239.1.1.1.pago-services1: [udp sum ok] UDP, length 1472

```

Comparing SEND_BUFFER_SIZE values

We can see that the output is exactly as expected (the + in the flags means the MF bit is set i.e. more fragments coming), with the 1472 byte send not being fragmented (no + in the flags) and is the reason why I have specified a 1472 byte max buffer size. Exceeding this means fragmentation, and **fragmentation is generally avoided in low latency systems for many reasons**, including increased complexity with the IP stack performing fragmentation, intermittent routers not supporting fragmentation (which can be avoided in controlled setups, such as HFT environments but this is still a negative), and the fact that losing a single fragment means the entire original IP datagram is lost. This was just a small experiment to confirm why fragmenting will be avoided here, it is better to performing application layer checks and keep the sending size within a single IP packet.

Now that the feature has been implemented, I decided to do one last benchmark and quantify the processing speed gained by batching the ITCH messages before sending. I compared the batching implementation to the independent one by running each one until they sent a total of 1,000,000 messages. To do this I implemented a global static counter and incremented on each call to the base Message class constructor. I did 10 runs of each and took the average, batches, also removing all logs and other noise.

<pre> ● menezesr@JM9X5HC4W4 benchmarking % ./benchmark_batched_sending === ITCH Generator Benchmark (Batched Sending)=== Running benchmark for 1000000 ITCH messages sent... === RESULTS === ITCH messages sent: 1000000 Total time taken: 721249250 ns Average time taken (per message sent): 721 ns Throughput 1.38648e+06 messages/sec_ </pre>	<pre> ● menezesr@JM9X5HC4W4 benchmark_independent_sending % ./independent_sending === ITCH Generator Benchmark (Independent Sending)=== Running benchmark for sending 1000000 batches ITCH messages... === RESULTS === ITCH messages sent: 1000000 Total time taken: 11375520666 ns Average time taken (per message sent): 11375 ns Throughput 87908.1 messages/sec_ </pre>
--	--

Batched vs Independent sending benchmarking output

Method of Sending	Total Time Taken (s)	Throughput (messages/s)	Time Taken / Message (ns)	Clock Cycles (3.2GHz)
Batching	0.582459667	89,803.4	582	1,862
Independent	9.964029084	1,480,520	9964	31885

Batched vs Independent sending for 1,000,000 messages

This shows that batching gives us a 17.12x increase over independent sending. This is a considerable improvement and serves as a good example of how small changes can make a huge difference if you know how the OS, microarchitecture and network interact with each other.

07/11/25

Today's focus was on the **implementation of sequence numbers** on the server side and tracking lost packets on the client side. This is where I got a bit of a taste in how things really work in production. Reading the ASX ITCH protocol spec, I found that there exists a re-request server, that the client can connect to via UDP unicast, providing the sequence number of the packet that was lost. The re-request server will then respond with that packet. Implementing a re-request server via unicast UDP would require storing all packets sent, which would require too much focus on the server side, while it is the client (MDFH) that needs to be optimized. I added it to the optimization list, however it was not a priority right now.

Adding the sequence numbers was quite simple, I just added a **global static variable (uint32_t so the numbers can wrap around, just like TCP)** and initialised it to 0 at the start of the program. I then added a pre-increment instruction for this global variable in the base Message class parametrized constructor, so every ITCH message created will increment this counter. I then updated the serialization functions by abstracting out the serialization of the messageType, timestamp and the new sequenceNumber to its own function for readability. That's all that needed to be done, though I do know that once I incorporate multithreading into my programs this will need to be updated as the sequenceNumber will be a shared resource that will need to be changed atomically.

I then shifted my focus to the client side to extend it to process batches of ITCH messages, instead of single independent messages. To do this, I modified the parsing loop to now iterate over the length of the buffer (output from recv) and parse each ITCH message, still based on the first byte of each message. Depending on this value, I know exactly how long the current ITCH message is, so I know how to parse the entire buffer. I used an enum to store the fixed sizes of each ITCH message (since it is static, using an enum makes the most sense as it is evaluated at compile time). I then updated the parsing and logging functions to process the sequence number. Looking at the output, I can see the batches of messages being parsed, and I can also see the retried message being the first message in the next batch.

```
timestamp: 43515070365000 | SN: 53
79265088 of XRP order added
timestamp: 43515070370000 | SN: 54
30790287 shares from order no: 1003 executed.
timestamp: 43515070373000 | SN: 55
1088769167 shares from order no: 1009 executed at price: 3434317886
timestamp: 43515070378000 | SN: 56
1348763712 of ETH trade executed
**MESSAGE WRITTEN TO RETRY BUFFER WITH SIZE: 36 bytes
Sent bytes: 1445
```

```
timestamp: 43521322645000 | SN: 57
2942895855 of ETH trade executed
timestamp: 43521322660000 | SN: 58
2380318297 shares from order no: 1010 executed.
```

Retry buffer from IMG being written to at SN = 56

```
[A] timestamp=43515070345000 sequenceNumber=48 orderRefNumber=1007 shares=247051629 stock=AAPL buySell=S price=2199888452 | 8023000 ns
[A] timestamp=43515070349000 sequenceNumber=49 orderRefNumber=1001 shares=3857614317 stock=DOGE buySell=S price=3717225619 | 8022000 ns
[A] timestamp=43515070354000 sequenceNumber=50 orderRefNumber=1005 shares=4076225509 stock=MSFT buySell=B price=574096285 | 8021000 ns
[S] timestamp=43515070358000 sequenceNumber=51 eventCode=[C] | 8020000 ns
[P] timestamp=43515070361000 sequenceNumber=52 orderRefNumber=1002 shares=2007990857 stock=META buySell=B price=2129741537 | 8020000 ns
[A] timestamp=43515070365000 sequenceNumber=53 orderRefNumber=1005 shares=79265088 stock=XRP buySell=S price=3470324032 | 8019000 ns
[E] timestamp=43515070370000 sequenceNumber=54 orderRefNumber=1003 executedShares=30790287 | 8017000 ns
[X] timestamp=43515070373000 sequenceNumber=55 orderRefNumber=1009 executedShares=1088769167 executedPrice=3434317886 printable=Y | 8017000 ns
BATCH PROCESSED
[P] timestamp=43515070378000 sequenceNumber=56 orderRefNumber=1009 shares=1348763712 stock=ETH buySell=S price=3144679935 | 6252841000 ns
[P] timestamp=43521322645000 sequenceNumber=57 orderRefNumber=1001 shares=2942895855 stock=ETH buySell=B price=2481995525 | 583000 ns
[E] timestamp=43521322660000 sequenceNumber=58 orderRefNumber=1010 executedShares=2380318297 | 566000 ns
[X] timestamp=43521322671000 sequenceNumber=59 orderRefNumber=1004 executedShares=2794642283 executedPrice=891923745 printable=N | 565000 ns
[A] timestamp=43521322676000 sequenceNumber=60 orderRefNumber=1001 shares=1623518148 stock=MSFT buySell=B price=3296307522 | 564000 ns
[A] timestamp=43521322680000 sequenceNumber=61 orderRefNumber=1007 shares=176591768 stock=VET buySell=B price=2267566349 | 563000 ns
```

First message from next batch in MDFH showing retry buffer message

From the above screenshots we can see that the message with sequence number = 56 was not written to the server side send buffer, as it would have exceeded the 1472 byte capacity. It was instead written to the retry buffer and then copied into the next send buffer (after flushing). On the client side we can see that in the top batch the message with sequence number = 56 is not there (last is 55) but it is there in the second batch as the first message. This indicates the retry mechanism is working correctly.

At this point I have a server capable of batching generated ITCH messages and sending it over UDP multicast, and a client capable of receiving the batched messages and parsing each one correctly. The server includes sequence numbers, that the client can use to detect packet loss (re-request server still a maybe at this point). Now the next course of action is the fun part; implementing multithreading and benchmarking everything. But before that I decided to **finalize all unit and performance testing** so that I can safely continue with multithreading, otherwise any bugs/bottlenecks I have will just become even harder to handle. I began with the IMG and included the batched and independent sending benchmarking programs in a new test folder and moved all main program files to an src folder. **I expect to make changes to my code here based on the results of the performance tests.**

I began with adding a unit test for testing the writing to the retry buffer, in the generateMessage() function. In all my unit tests I like to adopt a **GIVEN, WHEN, THEN framework**. Applying it here it means GIVEN a buffer that is almost full, WHEN we write an ITCH message that would exceed its capacity, THEN we expect the retry buffer to be written to. Conversely when the buffer is not full, writing an ITCH message should write to the normal buffer and the retry buffer should be empty. I adopted a similar approach for testing sequence number and timestamp monotonic increase and buffer alignment. I also made a similar test file for serialization of each ITCH message, asserting that the data members for each ITCH message object (created in the default constructor, which randomizes the values of the data members) is equal to the output of the implSerialize() functions of each class. Asserting this is important for ITCH message correctness before sending across the wire. I then moved onto the MDFH, adding tests for this was more straightforward, I just added a simple test for the parsing of ITCH messages.

The next thing I wanted to focus on was benchmarking. For IMG, I already did this previously when I benchmarked my batched vs independent sending implementation

before, with my batch implementation being chosen, so I moved on to the MDFH. To benchmark this I need to get the runtime for the parsing loop for a fixed amount of messages. I already have the IMG, which I could alter to send a fixed number of messages, but I wanted to simulate what's done in the real world, which is to **replay the bytes from historical data into the program**. This is deterministic data (unlike the IMG) so benchmarking this makes more sense. To implement this, I altered the IMG to output all its data to a large file. But now the problem was how would I get the bytes from this file into my parser benchmarking program? This is where the replay idea comes in, I can implement a **UDP replay server** that reads from the file and sends it to the multicast IP. It would be very similar to my IMG, except it uses a file to write to the 1472 byte buffer, rather than the generateMessage() function. The advantage here is that I don't need to read the bytes if it would exceed the buffer capacity, I just flush and read into the next buffer, so no retry buffer is necessary.

I configured the replay server to iterate over the file indefinitely, and in the benchmarking program I can set the number of messages I want it to parse by using a global counter, like the sequence numbers I implemented earlier. The UDP server was quite straightforward to implement and was able to see the output on my MFDH. I did notice that the one-way latency measurement is going to be from the timestamp read from the historical data, which will be the correct value but doesn't mean anything for parsing.

```

o menezesr@JM9X5HC4W4 replay_server % ./replay
=== ITCH MESSAGE UDP REPLAY SERVER ===
Socket initialised.
Socket options set.
Multicast destination address set.
Running UDP replay server...
[
(E) timestamp=3710316030000 sequenceNumber=3735 orderRefNumber=1002 executedShares=1849035987 | 1093444670000 ns
(S) timestamp=3710316030000 sequenceNumber=3736 eventCode=[O] | 1093444676000 ns
(E) timestamp=3710316031000 sequenceNumber=3737 orderRefNumber=1005 executedShares=2990667054 | 1093444681000 ns
(E) timestamp=3710316031000 sequenceNumber=3738 orderRefNumber=1005 executedShares=1896258816 | 1093444688000 ns
(P) timestamp=3710316032000 sequenceNumber=3739 orderRefNumber=1006 shares=2760276306 stock=SOL buySell=S price=1539430472 | 1093444813000 ns
(S) timestamp=3710316033000 sequenceNumber=3740 eventCode=[C] | 1093444827000 ns
(S) timestamp=3710316033000 sequenceNumber=3741 eventCode=[C] | 1093444832000 ns
(P) timestamp=3710316034000 sequenceNumber=3742 orderRefNumber=1010 shares=3276084940 stock=BTC buySell=B price=3956481669 | 1093444837000 ns
(E) timestamp=3710316035000 sequenceNumber=3743 orderRefNumber=1009 executedShares=3645982547 | 1093444843000 ns
(S) timestamp=3710316035000 sequenceNumber=3744 eventCode=[C] | 1093444847000 ns
(A) timestamp=3710316036000 sequenceNumber=3745 orderRefNumber=1006 shares=575838518 stock=SOL buySell=B price=4170925105 | 1093444852000 ns
(S) timestamp=3710316037000 sequenceNumber=3746 eventCode=[C] | 1093444856000 ns
(C) timestamp=3710316037000 sequenceNumber=3747 orderRefNumber=1001 cancelledShares=1330369898 | 1093444861000 ns
(X) timestamp=3710316038000 sequenceNumber=3748 orderRefNumber=1005 executedShares=2111335551 executedPrice=3159002810 printable=N | 1093444865000 ns
BATCH PROCESSED
]

```

UDP Replay Server running and bytes being read (deterministic output)

Now I needed to create a benchmarking program that times the parsing loop. This was also quite straight forward, I used a high resolution timer, just like when comparing batched to independent sending. I started the UDP replay server as a separate process and the benchmarking program as another and was able to get a measure for how long the parsing loop takes for 100,000.

```

=== RESULTS ===
Messages parsed: 100000
Time taken: 1065898083
Time taken per message: 10658
Throughput: 93817.600008 messages/sec

```

Parsing loop benchmark

I repeated this 10 times and got an average value of 10,351 nanoseconds. Another thing I realised is that this number includes a frequent `recv()` syscall, which initiates a syscall per packet received (which is a maximum of 1500 bytes). An idea I had was to batch ITCH messages in the socket receive buffer before issuing a syscall, so I can parse multiple packets at once. Like the batched vs independent sends, this would measure the batched vs independent reads. After some research I concluded I could replace `recv()` with `recvmsg()` which allows for receiving multiple messages just as I need. But the problem was that this was a Linux specific command, and as I was most recently working on a MacBook Pro, I needed to **switch setups to a Linux PC to continue benchmarking, but I will get to that tomorrow.**

This now gives me a good baseline to optimize on, I expect this number to increase considerably once I implement concurrency. I plan to have a one network thread that handles I/O and publishes the data to a thread safe queue, while having multiple parsing threads that handle processing of the packets from that queue. Benchmarking for this will be done after completing multithreading implementation.

08/11/25

My plan for today revolved around 3 things. Setting up the parsing device on a Linux machine so that I can optimize it to include batched reads of packets using `recvmsg()`, use a switched ethernet network and a wifi network as the link layer between the two devices and compare throughput and to **implement a way to count lost messages from the sequence numbers.**

I decided to begin with the lost messages implementation. I thought this would be simple, so I began implementing a function to store the most recent sequence number as the ITCH message was parsed and check if it was consecutive with the current sequence number. I stored this most recent sequence number in a global struct, as a static member and called the checking function in every parsing function.

```
inline void checkAndSetGlobalState(const uint32_t &seqNo) {
    if (GlobalState::lastSequenceNumber + 1 != seqNo &&
        GlobalState::lastSequenceNumber != 0) {
        GlobalState::lostMessages += seqNo - GlobalState::lastSequenceNumber;
    }
    GlobalState::lastSequenceNumber = seqNo;
}
```

Code snippet of first lost message count functionality

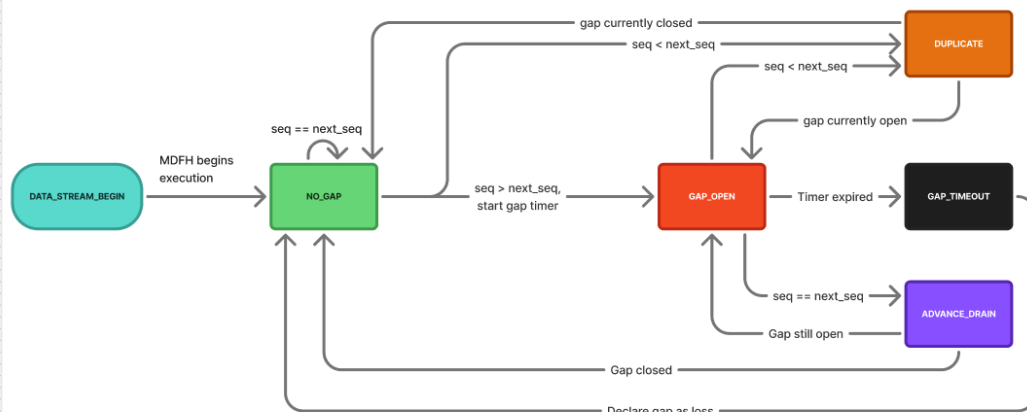
There are a few issues with this code. The first is that it does not consider the situation where the sequence numbers wrap around, causing `seqNo < GlobalState::LSN`. This means line 4 will evaluate to a garbage value (overflows to near $2^{32} - 1$, as it is a unsigned 32 bit integer). I also did not consider duplicates, which is when a message that has already been received is received again. Out of order messages is another situation I didn't think about here, just because the next sequence number is not consecutive doesn't mean the messages in between were lost. They may have just not arrived yet. UDP by design is not reliable, it is a best effort connection-less protocol and so all reliability is left up to the L7 application. **So I needed to come up with a way to handle these 3 issues: out of order messages, duplicates and lost messages.**

I did some research into how HFT firms handle these issues and got a few ideas. If the current sequence number is less than or equal to the last sequence number received IN ORDER, then we can conclude it is a duplicate. If it is anything larger than the next expected sequence number (which should be the last in order sequence number + 1) then we can conclude that it is received out of order (we can still parse it, but in reality this would depend on how the trading algorithm used interprets the data after parsing). In this case, a gap is created, where messages beginning from the next expected sequence number up until the current sequence number - 1 are possibly lost. They may just be in transit or queued at the NIC or kernel ring buffer and so time needs

to be given to allow these messages to come through. If after this timeout the messages still have not arrived, THEN we can conclude they are lost. Finally, if the current sequence number is the expected sequence number, then we increment the last in order sequence number seen. **Now the question is, how can we implement this?**

We probably need to have a global state, that keeps track of duplicates, out of order messages, lost messages, parsed messages and the **next sequence number expected**. That last one is key for concluding if a packet is a duplicate or not, the others are mainly for benchmarking. This number is also monotonically increasing, unless packets were concluded lost which means it must then jump to fill the gaps made by those lost packets. But how can we use this next expected sequence number to conclude whether a packet is a duplicate? **In TCP there is a term known as the “sliding window”**, which determines the window of sequence numbers that the sender sends/receiver expects. Any segments with sequence numbers outside this window are dropped, and the window opens (when the receive side application reads at a faster rate than the sender can acknowledge sent data), closes (when the send acknowledge rate is faster than the receive side read rate) or shrinks (when the acknowledge rate doesn't move, but the window size decreases which can happen if the receiver advertises a smaller window). **I can approximate this sliding window mechanism here**, where I keep track of all the recent sequence numbers received (sliding window) and it moves forward whenever the last in order sequence number advances. The window begins with the last received sequence number and ends with this number + window size. If the current sequence number is in this set OR lower than the lowest sequence number in the set (last in order sequence number), then it is considered a duplicate.

For out of order packets, if the current sequence number is larger than the expected one (last in order + 1) then a gap is created. In this case, a timer starts for the sequence numbers in that gap. The timer does not reset on every new gap created, but If they have not arrived by the time the timer expires, ALL packets in the gaps are considered lost and the last in order sequence number value jumps to fill the largest gap. **The sequence number that created the gap is treated as an out of order packet**. I spent some time organising my plan and creating a state diagram for this new sequencer I will be adding to the MDFH. Below is the result.



MDFH - Market Data Feed Handler, the name of the feed handler performing the parsing.

seq - the sequence number of the packet currently being processed.

next_seq - the next expected sequence number. This is a monotonically increasing value, meaning all sequence numbers before this have been received OR declared lost and it cannot increase for out of order packets.

NO_GAP - The initial state of the system once MDFH begins execution, and also the state when there are no out of order messages received.

1. If seq is equal to next_seq, the system remains in this state.
2. If seq < next_seq, transition to **DUPLICATE**
3. If seq > next_seq, an out of order message was received and a gap is created, transition to **GAP_OPEN**.

GAP_OPEN - If a packet was received out of order, the system transitions into this state where a "gap" of potentially lost sequence numbers is created. The gap exists in the current sliding window. The out of order counter is incremented and that packet is noted in the sliding window (and stored locally, but not processed to order book yet), but a timer also starts. Any sequence numbers still in the gap when the timer expires are considered lost and the system transitions into the **GAP_TIMEOUT** state. **NOTE: THE TIMER DOES NOT RESET WHEN ANOTHER OUT OF ORDER PACKET IS RECEIVED. THE ENTIRE WINDOW IS MARKED LOST (those not received) AND FLUSHED, AND THE SYSTEM RETURNS TO NO_GAP STATE.**

DUPLICATE - If seq is less than next_seq then we can conclude the received message is a duplicate. This is because next_seq does not advance unless every other contiguous sequence number prior to it has been received (exactly like ACKs in TCP). If a gap exists return to **GAP_OPEN**, otherwise return to **NO_GAP**.

GAP_TIMEOUT - Once the timer that began after entering **GAP_OPEN** expires, the sliding window is flushed and all sequence numbers not received are marked lost. The system then transitions into **NO_GAP**.

ADVANCE_DRAIN - When a gap is created, it is possible that next received packet has seq equal to next_seq. In this case, we must check the sliding window to see if we can increment next_seq further, as it is possible the gap may have closed. If sequence numbers remain in the window after cascading next_seq, transition back to **GAP_OPEN** otherwise transition to **NO_GAP**.

The state diagram makes implementation a lot easier, and so that is what I did next. I also kept track of the highest sequence number seen, which helps with concluding whether a gap still exists after **ADVANCE_DRAIN** or not. Implementing the duplicate (seq < nextSeq) and in-order (seq == nextSeq) logic was quite straightforward. It was when I got to the out-of-order (seq > nextSeq) logic that things got complicated. Entering the **GAP_OPEN** state means the gap timer needs to begin, and once expires we transition into the **GAP_TIMEOUT** state. But this **timer needs to run parallel to the main sequencing thread** (same as parsing thread currently) otherwise it will block parsing and sequencing. Furthermore, spawning a new thread every time a new out of order packet is detected is wasteful, as **there is significant overhead associated with repeatedly asking the kernel to create threads (creating a thread control block entry, scheduling the thread and then context switching from the process currently running on the chosen CPU to the thread).**

So I needed to come up with a better idea than just create the timer thread every time we transition from **NO_GAP** to **GAP_OPEN**. I also need to be very careful with ensuring atomicity over critical sections. My thought process here is to have the main thread set a gap start time, atomically and can exist in the global state struct, and the timer thread run indefinitely, parallel to the main thread, to constantly check if the timer expires, after which it must flush the bitset and increment lost messages accordingly. This means I need to mark lostMessages, gapStartTime and the bitset as atomic, to ensure only one thread has access to it at a time. Instead of using a synchronization primitive like a mutex or semaphore, which involves added overhead and resource contention

issues that can be and must be avoided in a low latency environment, I can opt for **lock-free programming** and use the inbuilt C++ assortment of atomic operations available in the `<atomic>` header, more specifically `std::atomic<T>`. These have simple `load()` and `store()` methods that I can use here.

The problem is that T only works for **lock free atomic types** and a bitset is not one of those. I have a couple options here. I can either change the bitset to be an array of `uint64_t` numbers OR just have the timer set flags that the main thread can then read and use to flush the bitset. I opted for the second approach as it is simpler because the bitset is modified only by the main thread, so no need for any atomicity. The timer thread will just set a flag in the `GlobalState` struct to signal that it has expired, the main thread will then check this and flush the bitset.

After implementing this design, I ran the MDFH locally and unsurprisingly found no lost messages, out of order packets or duplicates. The loopback of multicast traffic by default means it is very unlikely that any packets would be dropped. **I then ran the MDFH on another device on the same L2 network (WiFi) and the results were quite alarming at first. Both were run for a total of 100,000 ITCH messages being parsed and using a 4MB buffer.**

```
==== RESULTS ====
Messages parsed: 100000
Messages lost: 341538
Messages received out of order: 69490
Messages recieved as duplicates: 0
Time taken: 37601837709
Time taken per message: 376018
Throughput: 2659.444487 messages/sec
```

Parsing sequencer benchmarking output from another device

A whopping **341,538 messages lost** and a further **69,490 of the messages parsed were out of order**. This is quite typical of a Wi-Fi connection, especially considering there are many other devices in my home connected to the same WiFi AP and the fact that multicast is treated as broadcast by a WiFi AP, unlike a managed ethernet switch that uses IGMP snooping to figure out exactly which ports a multicast frame must be forwarded to. I plan to repeat this on a switched ethernet network in my next session. I didn't get around to setting up the MDFH on a Linux machine (to then use `recvmsg()`) or using trying out a switched ethernet network, but I will get to that tomorrow.

11/11/25

My first plan for today was to implement `recvmsg()` and fully switch to using Linux and use a switched ethernet network for the L2 technology of my project. This was necessary to compare the packet loss. To do this **I moved MDFH to the Linux device**, fixing up some of the code as it was not directly portable, such as needing to implement my own `ntohl()` function (converting a long long, 64 bit, integer from network byte order to host) and fixing up some preprocessor directives. After doing this I created a setup with my Mac device, running IMG, and my Linux device, running MDFH, each connected via the USB to ethernet interface to an unmanaged ethernet switch. I know that this means IGMP snooping is not used and so multicast will be treated like broadcast, but it is not really an issue here as there are only 2 ports active, so the setup behaves like unicast anyway. I may invest in a managed ethernet switch later and add more devices, but this is not necessary right now. Once the devices and switch were setup like this, I realised that I need to **configure my IMG to send the multicast packets out of the USB to ethernet interface**, and not the usual WiFi interface it was previously using. To do this, I simply overwrote the routing table the kernel uses to decide which interface to use for the 239.1.1.1 multicast IP by running the following command:

```
sudo route -n add -net 239.0.0.0/8 -interface en7
```

I ran `ifconfig` to confirm the USB to ethernet interface identifier as **en7**. I then ran IMG and `tcpdump` for en7, filtering for packets with the 239.1.1.1 destination IP and 30001 port in parallel to confirm that the multicast packets were routing through the correct interface now:

```
menezesr@JM9X5HC4W4 ~ % sudo tcpdump -i en7 -n host 239.1.1.1 and udp port 30001

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on en7, link-type EN10MB (Ethernet), snapshot length 524288 bytes
19:37:57.675016 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1462
19:37:58.680734 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1455
19:37:59.682935 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1444
^C
```

Output from tcpdump for en7 shows multicast packets routing through correctly

I needed to also do something similar on the Linux side to **configure the USB to ethernet interface to route in multicast packets**.

```
sudo ip route add 239.0.0.0/8 dev enx8a362d92729
```

I found the correct NIC identifier as `enx8a362d92729` and ran `tcpdump` on this first to confirm that the switch is working correctly.

```

roarkmenezes@dredge:~$ sudo tcpdump -i enx8a362d92729 -n host 239.1.1.1
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enx8a362d92729, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:08:43.765798 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1445
09:08:44.770080 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1454
09:08:45.773436 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1453
09:08:46.778483 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1465
09:08:47.783672 IP 169.254.193.250.63540 > 239.1.1.1.30001: UDP, length 1463
^C

```

USB to Ethernet NIC showing packets received

The next step then is to configure the kernel to route the multicast frames at the L2 layer (received by the USB to ethernet NIC) to the L3 layer. But to do this, the **interface needs to be given an IP address** so that IGMP membership report messages can be properly routed out. Otherwise my MDFH will fail when trying to set the IP_ADD_MEMBERSHIP socket option, because there is no route for these messages from the point of view of the kernel. I confirmed this by running `ip addr show` on this usb to ethernet interface, and saw no inet entry. There is no DHCP server in my switched ethernet setup and so this wont just be done magically. Setting a static IPv4 address is quite simple though, with the commands show below.

```

roarkmenezes@dredge:~$ ip addr show dev enx8a362d92729
11: enx8a362d92729: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel s
tate UP group default qlen 1000
    link/ether c8:a3:62:d9:27:29 brd ff:ff:ff:ff:ff:ff
roarkmenezes@dredge:~$ sudo ip addr add 192.168.10.2/24 dev enx8a362d92729
roarkmenezes@dredge:~$ ip addr show dev enx8a362d92729
11: enx8a362d92729: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel s
tate UP group default qlen 1000
    link/ether c8:a3:62:d9:27:29 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.2/24 scope global enx8a362d92729
        valid_lft forever preferred_lft forever
    inet6 fe80::552a:8422:812e:e3b8/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

Assigning an IP address to USB NIC

After doing all the above, the MDFH ran until the parsing loop, **however no packets were coming in**. I could see the NIC picking up the multicast frames, but no packets were arriving at my socket. I dumped the current multicast memberships for the NIC and confirmed that the 01:00:5e:01:01:01 MAC address was there (239.1.1.1 IP equivalent). I further diagnosed the problem by confirming that the NIC IP is there, I also altered the code to bind directly to this address but that didn't work. Then I noticed the below in the route to 239.1.1.1 IP (ran `ip route get 239.1.1.1`):

```

multicast 239.1.1.1 dev enx8a362d92729 src 192.168.10.2 rt_offload_failed uid 1000
cache <local,mc>

```

The `rt_offload_failed` tells me that the **NIC cannot offload the processing of L2 frames to the kernel**, which explains why my socket was not working. Instead of delving deeper and fixing the issue through assessing Linux internals, **I decided to**

optimize my code even further and bring it closer to what production grade HFT systems use: DPDK (Data Plane Development Kit). Normally, the NIC maps L2 frames to buffers in RAM (via DMA), which the kernel then reads and processes (stripping L2, L3, L4 and then finally delivering it to user space via the `recv()` syscall). **This entire process features many copies and context switches which are sources of latency that can be minimised. DPDK works to bypass the kernel entirely** and map the NIC DMA directly into user-space, giving the user program direct control over the L2 frame. This requires special drivers and setup but I will **use raw filtering of the ethernet frames i.e. read directly from the kernel buffers using a raw socket (AF_PACKET).** This also means that my parsing loop needs to strip the L2, L3 and L4 headers to get the UDP packet.

This means it also simplifies the socket creation part, I just bind it to a specific interface, no socket level options necessary (e.g. `SO_REUSEADDR`, `SO_REUSEPORT`) and no multicast joining required, I perform the frame filtering myself. Before beginning implementation, I planned out exactly what I needed to do. I first need to **bind the socket to the correct interface (which my Linux device uses for the multicast frames).** I previously found this to be `enx8a362d92729`. I simply created a link layer socket address (`sockaddr_ll`) with the `AF_PACKET` family and using the protocol for only IPv4 encapsulated ethernet frames and the index of the correct interface (which I found using the `if_nameindex()` library function).

```
sockaddr_ll socket_link_layer{};
socket_link_layer.sll_family = AF_PACKET; // AF_PACKET tells the OS to use this
socket for Ethernet frames, similar to AF_INET being for IPv4
socket_link_layer.sll_protocol = htons(ETH_P_IP); // Filtering is applied to
deliver only IPv4 encapsulated ethernet frames
socket_link_layer.sll_ifindex = index; // Set correct iface idx
```

```
bind(sockfd, (sockaddr *)&socket_link_layer, sizeof(socket_link_layer))
```

Code snippet for binding socket to correct interface

Now that the socket is bound to that interface, all L2 frames that show up there (that are `ETH_P_IP` type i.e. IPv4 encapsulated ethernet frames) will be copied into this socket buffer and `recv()` will pick them up. But this now means that I need to strip the ethernet header, IP header and UDP header of each packet, filter by destination IP address being the 239.1.1.1 multicast IP address, 30001 port, IP header protocol being UDP (17) and finally pass the UDP payload portion and size to the `parseMessage()` function.

```
while (1) {
    ssize_t nbytes = recv(sockfd, buf, sizeof(buf) - 1, 0);
    if (nbytes < 0) {
        perror("Error receiving data, terminating.");
        break;
    }
    iphdr* ip_header = (iphdr*)(buf + 14);
```



```

uint32_t dest_ip_addr = ip_header->daddr;
if (dest_ip_addr != inet_addr(MULTICAST_IP)) continue;
if (ip_header->protocol != 17) continue;

int ip_header_length = ip_header->ihl * 4;
udphdr* udp_header = (udphdr*)(buf + 14 + ip_header_length);
int udp_port = udp_header->dest;
if (udp_port != htons(PORT)) continue;

char *payload = buf + 14 + ip_header_length + 8;
ssize_t payload_length = ntohs(ip_header->tot_len) - ip_header_length - 8;
parseMessage(payload, payload_length);
fflush(stdout);
}

```

Code snippet of updated parsing loop

Each call to `recv()` will now give us an entire ethernet frame, which we then read into an ip header struct (advancing buffer by 14 bytes as that is the size of the ethernet header, as no VLAN tagging is involved here otherwise it would be 18 bytes. This probably needs to be considered here but because it is a simple project I will avoid the unnecessary complication). We then **filter based on the destination address in the ip header** (must be equal to the multicast IP 239.1.1.1 in network byte order) and by the protocol (must be equal to 17 which is for UDP). Then we obtain the **IP header length from the 4 bit IHL (internet header length) attribute in the header**, which gives us the header in 32 bit words, so we multiply by 4 to give us the total number of bytes). We use this to **obtain the offset of the UDP header** (original pointer + 14 + IHL * 4) and read this into a udp header struct. Now we can filter by the UDP port (must be 30001). Then, knowing that the size of a UDP header is 8 bytes, **we can now get a pointer to the UDP payload** (which will be original pointer + 14 + IHL*4 + 8). Obtaining the UDP payload size is done by getting the IP payload size (total length – IHL*4) and taking away the UDP header size (8). **We now have the updated buffer and buffer sizes to use for the parseMessage function.**

I created a new benchmarking program that benchmarks the raw socket with the new setup. I tweaked the receive buffer and NUM_MESSAGES sizes until I encountered a measurable loss of packets and out of order receiving and the results were extremely significant when compared to WiFi.

```

=== RESULTS ===
Messages parsed: 10000000
Messages lost: 39441
Messages received out of order: 58657
Messages recieved as duplicates: 0
Time taken: 7404784298
Time taken per message: 740
Throughput: 1350478.231041 messages/sec

```

Raw parser benchmarking output on Linux device

We can see that under a much higher load than the WiFi setup (and also using only a 64kB buffer compared to 4MB for the WiFi setup) we get a throughput of 1350478.231041, giving us a **507.804x increase**. The below table compares the WiFi setup to the ethernet

System Setup	Throughput (messages/s)	Loss rate	Out of order rate	Time taken per msg (ns)	Messages Sent	Receive buffer size (kB)	Duplicate rate
WiFi	2659.44	341.5%	69.49%	376,018	100,000	4096	0
Switched Ethernet Network	1,350,478.23	0.394%	0.5866%	740	10,000,000	64	0

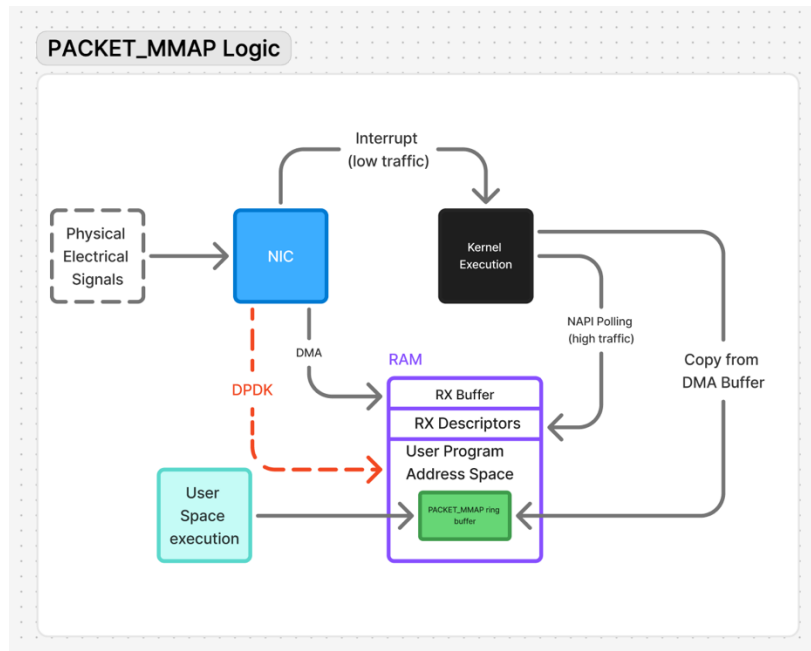
Table comparing WiFi setup to Ethernet

We can clearly conclude the switched ethernet setup is much more efficient with better numbers across the board.

Now we have the switched ethernet setup, the next point of optimization is to use `recvmsg()` instead of `recv()`. I thought about this for a while and realised that the current setup features 2 copies. One from the kernel to copy the data from the RX buffer (populated by the NIC via DMA) into the socket receive buffer and then another to copy from the socket receive buffer into the user space buffer (via `recv/recvmsg()`). But what if we could use only one copy and completely bypass the socket receive buffer as well? **So not only do we remove the socket receive buffer, but we remove one copy entirely and read directly from the RX buffer into user space.** This was possible with something known as **PACKET_MMAP frames (TPACKET)**. **There are 3 versions of this, I will first explore the first version.** I did some in depth research into how exactly the incoming ethernet frame received at the NIC is processed by the user space program to familiarise myself with the real process.

Normally, everything begins with the ethernet frame (with destination MAC address 01:00:53:01:01:01) arriving at the NIC. The NIC, via DMA, writes these frames to the RX ring buffer, completely independent of the CPU. Then, via an interrupt (or NAPI polling during high traffic), the kernel takes over and copies this data into its own pipeline to perform L2, L3 and L4 processing before copying it into the socket receive buffer for the `recv()` or equivalent to eventually copy it into user space. But with **PACKET_MMAP** frames, when the kernel is first notified to check the RX buffer, it copies the L2 frame into a **PACKET_MMAP** ring frame, which has its own header with a special flag called `tp_status`. This is a memory mapped (`mmap()`) region that we allocate ourselves, and it is shared between the kernel (writing the **PACKET_MMAP**'d frames) and user space (reading the frames). The key point here is that without synchronization of access to the shared ring buffer, race conditions are possible and thus we can get undefined behaviour. This is exactly why we need the `tp_status` flag, because it allows for a **lock-**

free ring buffer. The kernel sets the flag to **TP_STATUS_KERNEL** while it is loading the PACKET_MMAP frame, and sets it to **TP_STATUS_USER** once it is done. It then **NEVER** changes the status, the user space program is always given exclusive read access. The user space program then reads the frame and sets the status back to **TP_STATUS_KERNEL**, which the kernel can then read and overwrite with a new frame.



Packet flow diagram for PACKET_MMAP method

Here we have a packet flow diagram for how physical electrical signals transition to binary data in our user space program. The physical signals hit the NIC, it then uses the **preamble in the IEEE packet header to synchronize its incoming bitrate** to correctly process the message. The **NIC then polls the RX descriptors** (in RAM) to find a suitable **RX buffer to write to** and writes the encapsulated ethernet frames there (it can also cache these on the NIC itself for faster processing). **In times of low traffic, the NIC then issues an interrupt** to switch to kernel execution, which then queries the RX descriptors and finds the buffers that have available frames. **In times of high traffic, NAPI polling is used** to poll the RX descriptors until the ring is drained or its cycle budget is exceeded. In either case, the **frames from the RX buffer are encapsulated in a PACKET_MMAP header and copied into an mmap'd region in the user program address space**, where the user space execution can process it. This **removes the extra copy into the socket receive buffer** that is being used in my current implementation and can significantly improve throughput and also **NO NEED FOR RECVMMSG() AS WE ARE NOT USING A SOCKET RECEIVE BUFFER ANYMORE**. For comparison I have also labelled on the diagram what the **DPDK** flow looks like, the NIC maps the ethernet frames directly into user space, no kernel, no socket receive buffer, no copies, just direct DMA into user space.

The plan is now set and I can proceed with implementation, and I will compare the throughput with the `recv()` method. There isn't much change in terms of the code, the only part to change is creating the socket option for enabling the kernel to write to the new shared `mmap'd PACKET_MMAP` buffer instead of the socket receive buffer, and then iterating over this in our parsing loop instead of the socket receive buffer. We still need the same socket, the same bind to the multicast aware interface, and same frame parsing logic we currently have. So I began with setting up the socket to write to the `PACKET_MMAP` ring buffer. To do this we need to call `PACKET_RX_RING` `setsockopt()` on our socket, giving it a `tpacket_req` struct that sets the rings' **block size, frame size, block count and frame count**. Choosing these values is critical for finding a balance between memory usage, processing latency and lowering packet drop risk, as if MD5 cannot process the packets as fast as they come in, the kernel will drop packets as it will see all of them with the status `TP_STATUS_USER`.

Lets start with the frame size first. We know that the incoming ethernet frame is 1518 bytes (4 byte CRC is on the end and usually stripped but not 100% sure) and the `PACKET_MMAP` header is 32 bytes. That gives us a **total of 1546 bytes**, which we can easily align to a 64 byte cache line boundary of 1600 bytes. **I will use 2048** (for simplified memory management) and tweak the values after proper benchmarking later. **Frame count** is indicative of how many frames I want to be able to store at once. I will go with small number of 32 for now and tweak later. **This gives me a total buffer size of $2048 \times 32 = 64\text{kB}$ (same as original raw parser) and can easily fit in the L2 cache.** **Block size** is indicative of how many frames I would like the kernel to batch update. This is crucial in ensuring minimal cache misses and TLB misses, as with a batch load into the cache (and its equivalent VPN to PFN translation in the TLB) each frame in the block will fall in the same cache line (causing cache hits after the first load) and give a TLB hit as well. But its size is important, **as it cannot exceed the size of the caches**. Although the performance with a block size > cache size is still better than loading individual frames, it will cause consistent evictions of blocks from the cache to load new data and so is not as optimal as using block size <= cache size. Additionally, if a block spans more than 4kB (which is the span size for a single TLB entry i.e. it will cause a TLB hit for all virtual addresses in the range of that single TLB entry) it will need more than one TLB entry, which can be wasteful. On the other hand, if my block size is too small, that means I will incur the cost of context switches for each block for the kernel to load. My frame size already means I cannot fit it in the L1 cache, so I will use a size of **8 frames per block for now. I will need to use perf later to properly benchmark my program and optimize for minimal cache misses.** The block count is then simply the frame count divided by the block size, $512/8 = 64$ blocks.

Now we have our `tpacket` struct ready, I applied it to my socket via `setsockopt()`. **This call is what does the memory allocation for our shared ring buffer. The next step is to map this memory into our process address space, using `mmap()`.** This does not do

any memory allocations, it just updates the process page table to point to this shared ring buffer. Now, both the user space program and kernel are accessing the same shared memory, the kernel via its own page table entry (or TLB entry) and MDFH via its own PTE/TLB entry. **The next step was to completely remove the recv call, and now instead index the shared memory pointer and obtain the next frame on every loop iteration.** The frame would be in the form of a **TPACKET frame**, so we just cast it to this. Then we perform the last check which is to check that the frame tp_status flag is set to TP_STATUS_USER, otherwise we can **yield the CPU and let the scheduler assign another thread/process to the CPU**, because the frame isn't ready yet. We could just increment the frame counter and continue the loop but its very likely that if the current frame isn't ready then every other frame in the block isn't ready either, and because the kernel write sequentially, it is better to just wait until the current block is ready. But then I run the issue of **added latency from MDFH being scheduled back on the CPU and then its' context being switched back onto the CPU and continuing running the program.** It may be better to be selfish here and spin-wait until the frame is ready by constantly continuing, but I will perform that benchmarking later when I run perf. If the status is what we expect, then we simply obtain the ethernet frame from the packet (using the offset of the header size) and it goes straight into our previous packet processing code. No other changes required.

I added a few more optimizations such as removing the ethernet header length check and payload size check, and ran the benchmark. Below is the new code for the parsing loop:

```
uint16_t dest_port = htons(PORT);
for(uint32_t frame_num = 0; ;frame_num = (frame_num + 1) & (num_of_frames - 1))
{

    auto *frame_header = (tpacket_hdr *)((uint8_t*)ringPtr + (frame_num * frame_size));
    if (!(frame_header->tp_status & TP_STATUS_USER)) continue;

    char *buf = (char *)frame_header + frame_header->tp_mac;

    iphdr* ip_header = (iphdr*)(buf + 14);

    uint32_t dest_ip_addr = ip_header->daddr;
    if (dest_ip_addr != mcast_ip) {
        releaseFrame(frame_header);
        continue;
    }
    int ip_header_length = ip_header->ihl * 4;

    if (ip_header->protocol != 17) {
        releaseFrame(frame_header);
        continue;
    }

    udphdr* udp_header = (udphdr*)(buf + 14 + ip_header_length);
```

```

int udp_port = udp_header->dest;
if (udp_port != dest_port) {
    releaseFrame(frame_header);
    continue;
}

char *payload = buf + 14 + ip_header_length + 8;

ssize_t payload_length = ntohs(ip_header->tot_len) - ip_header_length - 8;
parseMessage(payload, payload_length);
handleGapTimeout();

releaseFrame(frame_header);
}

```

Code snippet of TPACKETv1 parsing loop

```

=== RESULTS ===
Messages parsed: 10000000
Messages lost: 6768
Messages received out of order: 6774
Messages recieved as duplicates: 0
Time taken: 6603029604
Time taken per message: 660
Throughput: 1514456.332884 messages/sec

```

TPACKETv1 parser benchmarking output

With the TPACKET logic, we get a throughput of 1514456.3329 messages per second, which provides a very significant **12.14% increase in throughput**.

System Setup	Throughput (messages/s)	Loss rate	Out of order rate	Time taken per msg (ns)	Messages Sent	Receive buffer size (kB)	Duplicate rate
TPACKETv1	1,514,456.33	0.068%	0.0677%	660	10,000,000	64	0
Raw parser	1,350,478.23	0.394%	0.5866%	740	10,000,000	64	0

Table comparing vanilla raw parser with TPACKETv1 raw parser

I was having a lot of fun optimizing this code, and was very satisfied to see that my knowledge of CPU caches, memory alignment, Linux kernel in the context of networking and C++ has paid off and I can see the gains in processing speed from **all the optimizations I made**. In my next session I am going to extend this to use TPACKETv3 and compare the gains.

14/11/25

Today I attempted an implementation of TPACKETv3. The biggest difference between TPACKETv1 and v3 is that in v1 the kernel fills the ring buffer frame by frame, but in v3 it fills the buffer block by block, with each one being **variable user determined size**. This means my original implementation of 2048 byte frames, which was only occupied by an approx. 1514 byte ethernet frame, was wasting considerable memory. When it comes to using a standard 4kB page size for the TLB entries, over many frames this will account for a large amount of TLB misses. If we could compact the frames so that we could get more TLB hits per set of frames (blocks) we could considerably improve processing speed. Additionally, there will be less context switches to the kernel as we are not populating the shared buffer frame by frame, but rather block by block. The same logic of checking the flag for the TP_STATUS_USER value applies **but most importantly the flag is set on the block, so we can check more frames before a context switch to the kernel is required to populate the next block.**

The block processing allows for other benefits including less loss sensitivity to bursts in the network. v1 means that once the ring buffer wraps around, all frames will have TP_STATUS_KERNEL and thus the kernel will begin dropping packets from the RX buffer, but with v3, the blocks can hold many packets at once. **Now when we traverse the shared ring buffer we iterate over full blocks, not frames.** I aim to once again benchmark the exact block size used based on performance, but these benefits were enough for me to conclude that this would optimize MDFH even further.

I went forward with planning my implementation by learning first how TPACKETv3 works. We will still be using a PACKET_RX_RING buffer that is mmap()'d into the MDFH address space. The same socket will bind to the same multicast aware interface. The only difference here is that the v1 API uses the block size as a means of aligning memory for loading data efficiently (directly into the L2 cache and caching the VPN to PFN translations in the TLB) while the **v3 API will iterate these blocks. We must also set the packet version to TPACKET_V3 on the socket, which is done by setsockopt() with a PACKET_VERSION equal to TPACKET_V3 at the SOL_PACKET level.** The mmap()'d region of memory is unchanged.

Then, instead of using tpacket_hdr pointers to iterate over the shared ring buffer region of memory, we use a **tpacket_block_desc pointer, which is a pointer to each block inside the region. Then each block will have frames with a tpacker3_hdr pointer.**

The results were a bit disappointing, I had not been able to exceed the throughput of TPACKETv1. I think I have hit a threshold in terms of throughput I can get for a single threaded program. Additionally, my parseMessage() function is being called per packet and already dominating the CPU, which could be the reason why I am capped at 1.4-1.5

million messages/sec. To confirm this I decided to begin analysing my program using **perf**.

```
Performance counter stats for './benchmark_raw_parser':

    11,478.11 msec task-clock                #    1.656 CPUs utilized
      140,468      context-switches          #    12.238 K/sec
        699      cpu-migrations              #    60.899 /sec
        646      page-faults                 #    56.281 /sec
  38,509,988,529 cycles                      #    3.355 GHz
  48,434,544,448 instructions                 #    1.26 insn per cycle
    7,790,794,251 branches                   #   678.752 M/sec
    41,344,694   branch-misses               #    0.53% of all branches

    6.932307402 seconds time elapsed

    10.288845000 seconds user
     1.326077000 seconds sys
```

```
Performance counter stats for './benchmark_tpacketv3':

    13,842.25 msec task-clock                #    1.984 CPUs utilized
      472      context-switches              #    34.099 /sec
        57      cpu-migrations               #    4.118 /sec
        646      page-faults                 #   46.669 /sec
  48,321,566,856 cycles                      #    3.491 GHz
  68,181,070,853 instructions                 #    1.41 insn per cycle
    9,609,663,563 branches                   #   694.227 M/sec
    19,664,088   branch-misses               #    0.20% of all branches

    6.976621408 seconds time elapsed

    13.297758000 seconds user
     0.543418000 seconds sys
```

```
Performance counter stats for './benchmark_packet_mmap':

    13,875.78 msec task-clock                #    1.991 CPUs utilized
      463      context-switches              #   33.368 /sec
        64      cpu-migrations               #    4.612 /sec
        645      page-faults                 #   46.484 /sec
  48,438,011,127 cycles                      #    3.491 GHz
  71,263,267,487 instructions                 #    1.47 insn per cycle
   10,013,130,161 branches                   #   721.627 M/sec
    23,114,373   branch-misses               #    0.23% of all branches

    6.970627619 seconds time elapsed

    13.090652000 seconds user
     0.784098000 seconds sys
```

Output from perf stat for raw parse, TPACKETv3 and TPACKETv1

From the above perf output we can see the differences between TPACKETv1/v3 and the base raw parser using `recv()`. **The raw parser had ~140,000 context switches, while the TPACKET implementations had only ~400.** This played a huge role in the throughput differences we saw earlier. One odd observation from the above is that the differences between TPACKETv1 and TPACKETv3 are not that noticeable, but both do seem to be having many branches and instructions. **Branches in general are sources of latency for programs due to misprediction penalties incurred from pipeline flushes in the even the branch prediction unit is wrong (as mentioned previously).** I decided to inspect further and see what I could find from the perf call graph, and I made

some interesting discoveries.

Children	Self	Command	Shared Object	Symbol
+ 49.04%	0.00%	benchmark_tpack	benchmark_tpacketv3	[.] std::__invoke_result<void (*)>::type std::__invoke<void (*)>((void (*)(*&))
+ 49.04%	0.00%	benchmark_tpack	benchmark_tpacketv3	[.] void std::__invoke_impl<void, void (*)>(std::__invoke_other, void (*)(*&))
- 48.36%	4.63%	benchmark_tpack	benchmark_tpacketv3	[.] gapTimer()
- 43.73%		gapTimer()		
+ 18.06%		std::chrono::duration<std::enable_if<std::is_convertible<std::enable_if<std::chrono::__is_duration<int>::value, int>::type const&, std::common_type<long,		
+ 13.08%		void std::this_thread::sleep_for<long, std::ratio<1l, 1000l> >(std::chrono::duration<long, std::ratio<1l, 1000l> > const&)		
+ 7.18%		std::atomic<bool>::operator bool() const		
+ 1.26%		std::is_constant_evaluated()		
0.59%		std::operator<=(std::strong_ordering, std::__cmp_cat::__unspec)		
0.56%		auto std::chrono::operator<=<long, std::ratio<1l, 1000l>, long, std::ratio<1l, 1000l> >(std::chrono::duration<long, std::ratio<1l, 1000l> > const&, std::ch		
0.52%		std::chrono::duration<long, std::ratio<1l, 1000l> >::count() const		
+ 4.63%		clone3		
+ 19.28%	16.77%	benchmark_tpack	benchmark_tpacketv3	[.] std::chrono::duration<std::enable_if<std::is_convertible<std::enable_if<std::chrono::__is_duration<int>::
+ 14.50%	3.12%	benchmark_tpack	benchmark_tpacketv3	[.] void std::this_thread::sleep_for<long, std::ratio<1l, 1000l> >(std::chrono::duration<long, std::ratio<1l,
+ 12.32%	1.14%	benchmark_tpack	benchmark_tpacketv3	[.] parseMessage(char const*, long const&)
- 11.18%		parseMessage(char const*, long const&)		
+ 2.48%		parseTrade(char const*, TradeMessage&)		
+ 2.25%		parseOrderWithPrice(char const*, OrderExecutedWithPriceMessage&)		

Call graph from perf report for benchmark_tpacketv3

From the above I can see that the gapTimer() function, that's run on separate thread as part of my sequencing logic to detect and resolve messages received out of order, has child functions (functions called within the gapTimer() function) that are consuming 48.36% of the CPU. Digging deeper, we can see that this is mostly due to the std::chrono::duration function, which I use to detect whether the or not the timer has expired, and the this_thread::sleep_for() function, which I use to sleep the thread so it does not spin wait. But upon inspecting the code, I can now see how wasteful this is. Instead of continuously checking if the timer has expired by constantly recomputing the current timer value, we should **let the main thread notify the timer thread via a condition variable**. This should eliminate the need for the std::chrono::duration call altogether. **This, along with fixing the other synchronization bugs in the sequencer, is my next task. I will then look to enable multithreading for the parser.**

17/11/25

First order of business today is to **ensure synchronicity of the sequencer**. I've identified the critical sections which are handling the gap timeout i.e **handleGapTimeout()** and the entire logic to update the GlobalState struct data members, depending on if duplicates, out of order or expected sequence numbers were detected. The thing is that locks themselves (and other synchronization primitives) carry overhead (object size + data members, locking and unlocking etc.) and I already have incorporated lock-free programming into my solution. I just need to extend it to include ALL members of the struct, including the seen bitset which is currently being written to by both the checkAndSetGlobalState() function (called in each parseMessage() function) and the handleGapTimeout() function (called whenever the timeout expires and ADVANCE_DRAINS the bitset marking all unseen, 0, bits as lostMessages). I also need to extend it to reduce the CPU usage of the gapTimer() function as mentioned at the end of the last log entry.

The problem here is that a bitset is not a usable lock-free atomic type, so I need to use something else. The simple fix here is to convert the std::bitset<WINDOW_SIZE> to an std::atomic<uint8_t> array. Now we can use the atomic load/store instructions to guarantee atomicity when modifying this array. For the other atomic variables, some of them are based on if statements. For example, the below:

```
if (GlobalState::nextSeq == 0) GlobalState::nextSeq = seq;
```

To make this work with concurrency, **we use something called a compare and swap (CAS) instruction**. This is guaranteed to be atomic, across all cores, and in C++ we can use the **compare_exchange_strong()** or **compare_exchange_weak()** instructions. The first guarantees to either set or not set the operand based on the given expected value. The second may spuriously fail, and so is frequently used in a spin loop. **Depending on the CPU, strong may be more efficient than weak + spin wait**. The expected value in the function asserts that the expected value must be an lvalue, so we cannot pass in an rvalue directly, we must first create a variable and store the expected value, and then pass it in.

But before doing this, I realised that my logic for draining the seen array isn't working properly. When the set wraps around, old bits marking old sequence numbers aren't being reset. One way to fix this is to reset the array to 0 every time we wrap around, but this is wasteful for a 65536 bit array. Another way is to store the sequence number that set that element in the array, then when draining we check whether the value is equal to the expected sequence number. That means the above line of code then becomes:

```
uint32_t expected = UINT32_MAX;  
GlobalState::nextSeq.compare_exchange_strong(expected, seq);
```

So the general rule is that if setting the value is based on a comparison, then use CAS, if it is unconditional (e.g. post increments or value setting) use store(). For making increments work with concurrent lock-free structures, we can use **fetch_add()**.

To reduce the CPU usage of the gapTimer() function I initially planned to use a CV that would be signalled when the gap opened, and the gapTimer() function would then start the timer and after it expires run the handleGapTimeout() function. But after some research I found out that **CV's are kernel mediated, meaning the underlying pthread_cond_wait() function called when calling cv.wait() is not immediately woken up when signalled. The main thread calling pthread_cond_signal() requires another syscall to kernel to reschedule the waiting thread. This is wasteful.** What I need to do here is pin the gapTimer thread to an isolated core so that it does not interfere with the main thread. There isn't much else I need to change.

Another aspect of optimization that I had planned for earlier was **CPU pinning. By setting the "CPU affinity" for a certain thread to A SINGLE CORE, the scheduler will assign that thread only to that core, allowing to build up a state in that cores' caches (TLB, L1, L2, L3) and thus remove the possibility of CPU migrations.** This also reduces the number of cache misses and TLB misses that would otherwise occur if that thread was scheduled on another core that didn't have the threads instructions/data and VPN to PFN translations sitting in the caches/TLB.

One other neat trick I thought of was to monopolize the CPU by **setting the scheduling priority to the maximum possible value**, meaning it will pre-empt all other tasks that Linux normally schedules via CFS. These are what we call **SCHED_OTHER** tasks, and they accumulate vruntime as they run on the CPU (for a dynamic time slice that is based on their nice value). **My idea is to set the category for the timer thread to be SCHED_FIFO, meaning it is scheduled solely based on priority, and by setting it to the highest value**, it should pre-empt all other SCHED_OTHER tasks and only be in contention with other highest priority SCHED_FIFO tasks.

Running perf now we can see the updated results

```
Performance counter stats for './benchmark_tpacketv3':

      13,448.64 msec task-clock                #    1.941 CPUs utilized
         158      context-switches            #    11.748 /sec
           2      cpu-migrations              #     0.149 /sec
        200      page-faults                 #    14.871 /sec
  34,206,331,563      cycles                  #     2.543 GHz
  45,109,366,486      instructions            #     1.32  insn per cycle
  6,821,040,070      branches                 #   507.192 M/sec
  14,549,288      branch-misses              #     0.21% of all branches

 6.928510125 seconds time elapsed

13.443205000 seconds user
 0.006996000 seconds sys
```

Perf stats for updated sequencer

From the above we can see that now there are only 2 CPU migrations, which the Linux kernel can do despite CPU pinning. **Non-uniform memory access (NUMA) is a type of architecture that localizes portion of memory to each core.** These “nodes” are physically attached to each core, meaning memory access from that specific core is much faster than all other nodes. To take advantage of this, the scheduler may move memory into the node specific to the current core OR it may move it away which causes TLB misses/cache misses. To avoid this, **its important to allocate memory with “NUMA awareness”.** I have added this as an optimization for later, but I achieved my goal for today. The next order of business is to extend MDFH to incorporate multithreading. How that will be done will be brainstormed and discussed in my next entry

20/11/25

Today I brainstormed and researched ways **to implement multithreading to enhance the throughput of the MDFH, while also reducing latency**. I had to keep in mind that the last part of the project was implementing writing to a KDB table, via IPC. With the current setup, the `parseMessage()` function acts as a bottleneck because it is called on each frame inside the `tpacket_v3` block. While parsing occurs, the extraction of the next frame's UDP payload is stopped, as the program is single threaded. What if we can decouple the parsing from the UDP payload extraction? This is the gist behind my current idea, which is to have a single "network thread" that reads from the shared `mmap'd` region (populated by the kernel via copying from the NIC RX buffer, which is populated via DMA), extracts the UDP payload and copies it into a dedicated queue that is handled by a certain parsing thread. This is then repeated for the next parsing thread, capped at `std::thread::hardware_concurrency()`, which on my machine is only 4. The setup then basically becomes **4 single producer, single consumer (SPSC) queues**.

Now the question becomes, how can we ensure synchronicity? Using locks is valid however, as always, it comes with added overhead with locking/unlock mutexes, using `std::unique_lock`/other RAIL locking objects costs precious CPU cycles. Instead we can opt for a **lock-free SPSC queue**. But how? I did some investigation into this, and **one way would be to use a flag**, like how `TPACKET` uses the `tp_status` flag (`TP_STATUS_USER/TP_STATUS_KERNEL`). This means that each node in the queue would need to have this flag along with the attached data (which in this case is the UDP payload data). The problem with this implementation is that the network thread and parsing thread run on different cores. If each are accessing the same memory, the **cache line containing the flag will bounce between the cores**, due to constant invalidations each time the cache line is modified by each thread. This is unavoidable as the network and parser threads must run on different cores, and that's just by design.

After realising this, **I remembered that my TPACKETv3 implementation did not bind the IRQ handler that handles setting the `tp_status` flag on the block after all the frames are populated to the same core as the user space thread**. This means massive invalidations will occur every time the cache line is modified by the kernel and user space thread. I've added this as an optimization after implementing multithreading.

I needed to come up with another solution. After doing some investigation, I found a better technique that uses indices, each for the read (consumer) and write (producer). With this design, there is no need for a flag on every node in the queue, synchronicity is ensured by the indices themselves because the producer only ever modifies the write index and the shared buffer, while the consumer only ever modifies the read index. When the consumer reads from the buffer, it will never be in contention with the producer, because of the way the write and read indices work. **The write index and read index will only ever be equal if the queue is empty, and this is always checked before reading**.

If we align the write and read indices to their own cache lines (64 byte padding) we remove the possibility of false sharing and thus cache invalidations. This means we have the means to implement a lock free SPSC queue without any cache invalidations, a much better solution than using the flags.

If the queue is empty, the parsing thread (consumer) cannot read i.e. pop() not allowed. If the queue is full, the network thread (producer) cannot write i.e. no push() is allowed. But how can we check if the queue is empty/full. A naïve implementation will use the modulus operator, but this is an extremely expensive operation. **Modern compilers however can optimize this to a bitwise AND operation only if modulo operand is a power of 2.** This is orders of magnitude faster than a modulo operation, but this means the queue capacity must be a power of 2. This is standard however, and I am used to working with this constraint such as with the frame and block sizes used for TPACKETv3. The way it does this is by **using a mask, equal to the capacity – 1**. Take for example the push() operation. We first check if the queue is full, by checking if (write + 1) % capacity = read. This is equivalent to (write + 1) & mask = read, a much faster operation. Getting the current size of the queue is done similarly, and because of the uint32_t type in combination with using the mask, wrap arounds are perfectly taken care of. With this, I went ahead and implemented the SPSC queue class.

```
template <typename T>
class SPSCQ {
    std::atomic<uint32_t> _write_idx{0};
    std::atomic<uint32_t> _read_idx{0};
    const size_t _capacity{0};
    const size_t _mask{0};
    std::unique_ptr<T []> _buf;

public:
    SPSCQ(size_t capacity): _capacity(capacity), _mask(capacity - 1) {
        if ((capacity & (capacity - 1)) != 0) throw std::runtime_error("Failed to
construct SPSCQ: Capacity must be a power of 2");
        if (capacity < 2) throw std::runtime_error("Failed to create SPSCQ: Capacity
must be at least 2");
        _buf = std::make_unique<T []>(capacity);
    }
    SPSCQ(const SPSCQ&) = delete;
    SPSCQ& operator=(const SPSCQ&) = delete;
    SPSCQ(const SPSCQ&&) = delete;
    SPSCQ& operator=(const SPSCQ&&) = delete;

    [[nodiscard]] bool full() {
        uint32_t w = _write_idx.load(std::memory_order_relaxed);
        uint32_t r = _read_idx.load(std::memory_order_acquire);
        if (((w + 1) & _mask) == r) return true;
        return false;
    }
}
```

```

[[nodiscard]] bool empty() {
    uint32_t w = _write_idx.load(std::memory_order_acquire);
    uint32_t r = _read_idx.load(std::memory_order_relaxed);
    if (w == r) return true;
    return false;
}

[[nodiscard]] size_t size() {
    uint32_t w = _write_idx.load(std::memory_order_acquire);
    uint32_t r = _read_idx.load(std::memory_order_acquire);
    return ((w - r) & _mask);
}

bool push(const T& item) {
    if (full()) return false;
    uint32_t w = _write_idx.load(std::memory_order_relaxed);
    _buf[w] = item;
    _write_idx.store((w + 1) & _mask, std::memory_order_release);
    return true;
}

bool pop(T& item){
    if (empty()) return false;
    uint32_t r = _read_idx.load(std::memory_order_relaxed);
    item = _buf[r];
    _read_idx.store((r + 1) & _mask, std::memory_order_release);
    return true;
}
};

```

My implementation of a SPSC queue

Now that I have a SPSC queue implementation, **the next step is to configure the network thread to write to these queues**. I needed to think carefully about how to implement this. What exactly should the queue store? If it was to store the UDP payload directly, that would involve a copy for each time we call push/pop. This is a colossal waste of memory bandwidth, it would be better to use pointers instead. But if I store pointers, then the UDP payloads themselves need to be copied somewhere first. I can't use pointers to the shared mmap()'d buffer, because the block cannot be released until all frames have parsed. This couples the network and parsing threads, which completely voids concurrency. **A copy from the network thread is unavoidable, and it ensures concurrency can still occur.**

This means that I would need to copy the UDP payloads from the network thread (producer) into a separate buffer first, then store the pointers to these buffers in the parser threads (consumer) queues. But this buffer must be managed carefully, because it cannot be overwritten by the network thread before the parser threads have had a chance to parse the messages. How can I manage this? We could use a flag for each

payload, just like the TP_STATUS_KERNEL/TP_STATUS_USER flags used with TPACKET. The network thread sets the flag (to user) and the parser thread processes the flag and reverts it (back to kernel). The problem with this is both the parser and network threads would need to iterate the buffer looking for slots to populate/process. I thought of a way to avoid this by using the SPSCQ class I just created. **We could instead initialize a SPSCQ for each parser thread that contains pointers to “free” slots in the buffer,** those that have either been processed already or genuinely free (as is on first use, like compulsory cache misses). The network thread pops from this queue, copies the UDP payload from the shared mmap()'d buffer (after stripping L2, L3, L4 headers) and then pushes the pointer to the data and length into the parser threads SPSCQ. **Now the parser threads work on a lightweight queue that contains only a pointer and a size_t length, on a 64 bit system this is only 16 byte.** The only thing here is that each parser thread needs its own incoming SPSCQ (for buffers to be parsed) and outgoing SPSCQ (for buffers that are free), which means 2 queues per parsing thread.

Now the question remains, how should the network thread pop from each parser threads outgoing queue (free buffers)? **Just like how we push the buffer to parse into the first available queue, we can also pop from the first available free buffer queue.** All I need to do now is initialize the queues of free slots (for each parsing thread), by first initializing the entire buffer used to store all UDP payloads and then storing pointers to slots of this buffer in the free queues. **I created a new BufferPool class to handle creation of the contiguous memory for storing the buffers on the heap,** the parserQueues and freeQueues and the pushing of all the buffer pointers into the free queues (equally, based on the number of parsing threads).

I then created the parsing thread function, which takes in its own parsing queue, free queue and cpu number, used to pin it to a specific core. The function simply pops a buffer from its parsing queue (populated by the network thread), runs parseMessage() on it, and then pushes the buffer back into the free queue, so the network thread can reuse it. The last thing to do now is to pin the IRQ handler populates the TPACKETv3 blocks and sets the tp_status flag to the same CPU as the network thread, so there's no cache line bouncing between CPUs, as cache invalidations cost a lot of time.

Running the program I could see the parsed messages coming in, which means everything was working correctly, however I needed to benchmark the program to confirm that there is a significant gain using this over a single threaded TPACKETv3 implementation. I ran perf on my benchmarking program and found some puzzling results. But it later made sense after I ran **lscpu** to confirm the number of physical cores vs logical cores, NUMA nodes and cache sizes. Upon inspection of the results I concluded that in terms of software optimization I had hit a roadblock, as the underlying hardware on my Linux system would not allow me to get any higher throughput. From

here I needed to make optimizations to the hardware.